

The University of Faisalabad

Lab Manual

Subject:

Data Mining

By

Abdul Haseeb Arif

2023-BS-AI-033

Semester:

6th

Course Code:

DS-303

Supervised by:

Mr.Arham

The University of Faisalabad

Bachelors of Science in Artificial Intelligence

DEPARTMENT OF COMPUTER SCIENCE

Sr. No.	Lab Title	Page No.
1	Lab 1: KNIME Introduction	1
2	Lab 2: Customer Churn Prediction	12
3	Lab 3: Data Cleaning	18
4	Lab 4: Data Preprocessing	23
5	Lab 5: Market Basket Analysis	30
6	Lab 6: Apriori Algorithm (Product Combination Analysis)	35
7	Lab 7: FP-Growth (Efficient Frequent Pattern Mining)	40
8	Lab 8: Python Basics (Reusable Data Processing Module)	45
9	Lab 9: Decision Tree	50
10	Lab 10: Naïve Bayes & KNN	56
11	Lab 11: Support Vector Machine (SVM)	61
12	Lab 12: Clustering	67
13	Lab 13: Outlier Detection	72
14	Lab 14: Web Scraping and Sentiment Analysis	74
15	Lab 15: Final Capstone Project	76

Lab 1: KNIME Introduction

➤ **Real World Problem:**

A retail business has accumulated transactional sales data spanning multiple product categories. However, raw data alone offers little actionable insight without proper analysis. The management requires structured analytical results to assess total revenue performance, understand average customer order spending, determine which categories generate the highest income, and pinpoint the most profitable products.

➤ **Dataset:**

The dataset used in this lab is a Retail Sales CSV file containing transaction-level records.

Each row corresponds to a single sales transaction and includes relevant attributes such as:

- Transaction ID
- Product Name
- Product Category
- Quantity
- Price per Unit
- Order Date
- Total Amount (or Revenue calculated during analysis)

The dataset structure allows aggregation, comparison, and ranking of sales performance across different dimensions.

➤ **Software Environment:**

The Software Environment used in this Lab is KNIME

➤ **KNIME Analytics Platform**

KNIME is a visual, node-based data analytics tool that enables users to build structured workflows for data processing and analysis. Each node performs a specific operation such as reading data, transforming attributes, computing statistics, or aggregating values.

The connected nodes together form a complete analytical pipeline that converts raw data into meaningful business metrics.

➤ Practical Objectives:

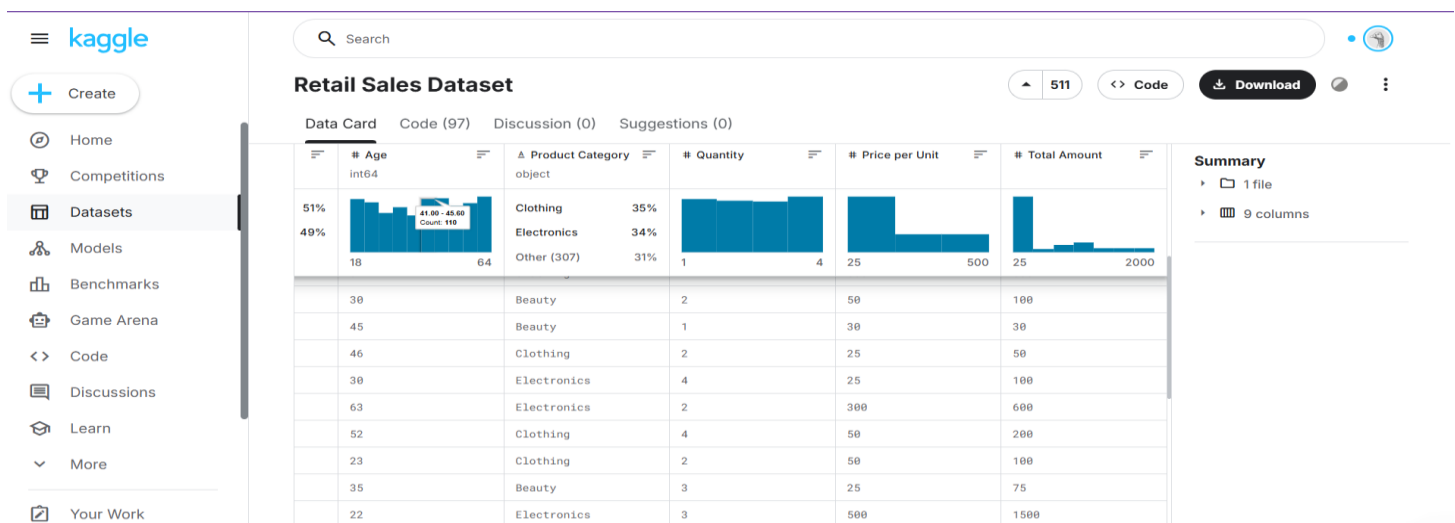
By completing this lab, students will gain practical experience in using KNIME for structured data analysis. Specifically, students will be able to:

1. Import a CSV dataset into a KNIME workflow using the CSV Reader node.
2. Examine and correct column data types to ensure computational accuracy.
3. Generate descriptive statistical summaries to understand dataset characteristics.
4. Create a calculated Revenue column using the Math Formula node.
5. Compute four key retail performance indicators using GroupBy and Sorter nodes.

➤ Procedure:

➤ Step 1: Download the Dataset:

Obtain the Retail Sales dataset in CSV format from the provided source and save it locally for analysis.



➤ Step 2: Create a New Workflow

Launch KNIME Analytics Platform and create a new workflow named *Retail Sales Analysis* to organize the analytical process.



➤ Step 3: Import the Dataset:

Use the CSV Reader node to load the Retail Sales dataset into the workflow environment and verify successful execution.

CSV Reader

Reads CSV files. To auto-guess the file click the Autodetect format button. If you encounter problems with incorrect types disable the Limit data rows in the Advanced Settings tab. If the changes between different invoices Support changing file schemas in the Advanced Settings tab. For further information see the KNIME File Handling Guide [File Handling](#).

Note: If you find that this node causes issues, try switching to the **File Reader** node. It offers more flexibility for reading complex files.

This node can access a variety of file systems. More information about file handling is found in the official [File Handling](#) guide.

Parallel reading: Individual files can be read in parallel if:

- They are located on the same machine or network drive.
- They don't contain any quote delimiters.
- They are not gzip compressed.
- No lines or rows are limited or filtered.
- The file index is not prepended.
- They are not encoded with UTF-16BE (is fine).

Settings | Transformation | Advanced Settings | Limit Rows | Encoding | Flow Variables | Job Manager Selection | Memory Policy

Input location

Read from: **Local File System**

Mode: ☒ File ☐ Files in folder

File: **Browse...**

Reader options

Format

Settings

Column delimiter: Row delimiter: ☒ Line break ☐ Custom

Quote char: Quote escape char:

Comment char:

☒ Has column header ☐ Has RowID

☐ Support short data rows ☐ Prepend file index to RowID

Preview

The suggested column types are based on the first 10000 rows only. See 'Advanced Settings' tab.

Row ID	Transac...	Date	Custom...	Gender	Age	Product ...	Quantity	Price pe...	Total A...
Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150
Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000
Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30
Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500
Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100
Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30
Row6	7	2023-03-13	CUST007	Male	46	Clothing	2	25	50
Row7	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100
Row8	9	2023-12-13	CUST009	Male	63	Electronics	2	300	600
Row9	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200
Row10	11	2023-02-14	CUST011	Male	23	Clothing	2	50	100

OK Apply Cancel ?

➤ Step 4: Verify the Data Types

Inspect all columns to ensure numeric fields (Quantity, Price, Revenue) are correctly typed as Integer or Double, and categorical fields (Category, Product Name) are set as String.

► 1: File Table Flow Variables

Rows: 1000 | Columns: 9 Table Statistics

#	RowID	Transacti...	Date	Customer ...	Gender	Age	Product C...	Quantity	Price per ...	Total Amo...
		123 Number (I...	T String	T String	T String	123 Number (I...	T String	123 Number (I...	123 Number (I...	123 Number (I...
1	Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150
2	Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000
3	Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30
4	Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500
5	Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100
6	Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30

➤ Step 5: Generate Summary Statistics:

Apply the Statistics node to compute descriptive measures such as mean, minimum, maximum, sum, and standard deviation for numeric attributes.

Column filter

Manual Wildcard Regex Type

Search Aa

Excludes

Includes

- Transaction ID
- Date
- Customer ID
- Gender
- Age

Discard Apply and Execute

► 1: Statistics Table ► 2: Nominal Histogram Table ► 3: Occurrences Table Flow Variables

Rows: 5 | Columns: 16

Table Statistics

	#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurtosis	Overall su...	No. n
	1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2	500,500	0
	2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201	41,392	0
	3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393	2,514	0
	4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139	179,890	0
	5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815	456,000	0

➤ Step 6: Calculate Revenue Column:

Use the Math Formula node to create a new Revenue column using the formula:

$$\text{Revenue} = \text{Quantity} \times \text{Price}$$

This derived column will be used for KPI calculations.

Dialog - 3:3 - Math Formula

File Edit Execute Cancel

Math Expression Flow Variables Job Manager Selection Memory Policy

Column List

- ROWINDEX
- ROWCOUNT
- Transaction ID
- Age
- Quantity
- Price per Unit
- Total Amount

Function

- pi
- e
- COL_MIN(col_name)
- COL_MAX(col_name)
- COL_MEAN(col_name)
- COL_MEDIAN(col_name)
- COL_SUM(col_name)
- COL_STDEV(col_name)
- COL_VAR(col_name)
- ln(x)
- log(x)
- log10(x)
- exp(x)

Expression

1 \$Quantity\$ * \$Price per Unit\$

Append Column: new column

Replace Column: Total Amount

Convert to Int

OK Apply Cancel

► 1: Group table Flow Variables

Rows: 1 | Columns: 1

☐	#	RowID	Sum(Quantity) 123 Number (Integer)
☐	1	Row0	2514

➤ Step 7: Calculate Total Revenue (KPI 1 – Total Revenue):

Add a GroupBy node without selecting any grouping column and aggregate the Revenue column using the Sum function to compute overall sales revenue.

Dialog - 3:4 - GroupBy

File

Settings Description Flow Variables Job Manager Selection Memory Policy

Groups Manual Aggregation Pattern Based Aggregation Type Based Aggregation

Aggregation settings

Available columns

- Transaction ID
- Date
- Customer ID
- Gender
- Age
- Product Category
- Quantity
- Price per Unit
- Total Amount

Select

add >>

add all >>

<< remove

<< remove all

To change multiple columns use right mouse click for context menu.

Column	Aggregation (click to change)	Missing	Parameter
Total Amount	Sum	☐	

Advanced settings

Column naming: Aggregation method (column name) ☐ Enable hiliting ☐ Process in memory ☐ Retain row order

Maximum unique values per group 10,000 Value delimiter ,

OK Apply Cancel ?

➤ Step 8: View Total Revenue Result:

Execute the node and observe the single output value representing total revenue across all transactions.

► 1: Group table Flow Variables

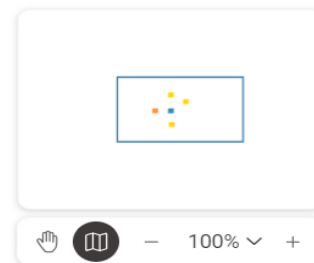
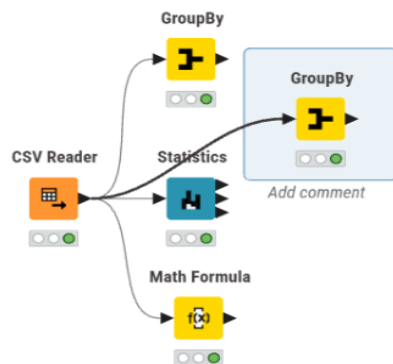
Rows: 1 | Columns: 1

Table Statistics

☐	#	RowID	Sum(Total Amount) 123 Number (Integer)
☐	1	Row0	456000

➤ Step 9: Calculate Revenue per Transaction

Add a GroupBy node, group by Transaction ID, and aggregate Revenue using Sum to compute total revenue for each individual order.



This node dialog is not s

Open dialog

► 1: Group table Flow Variables

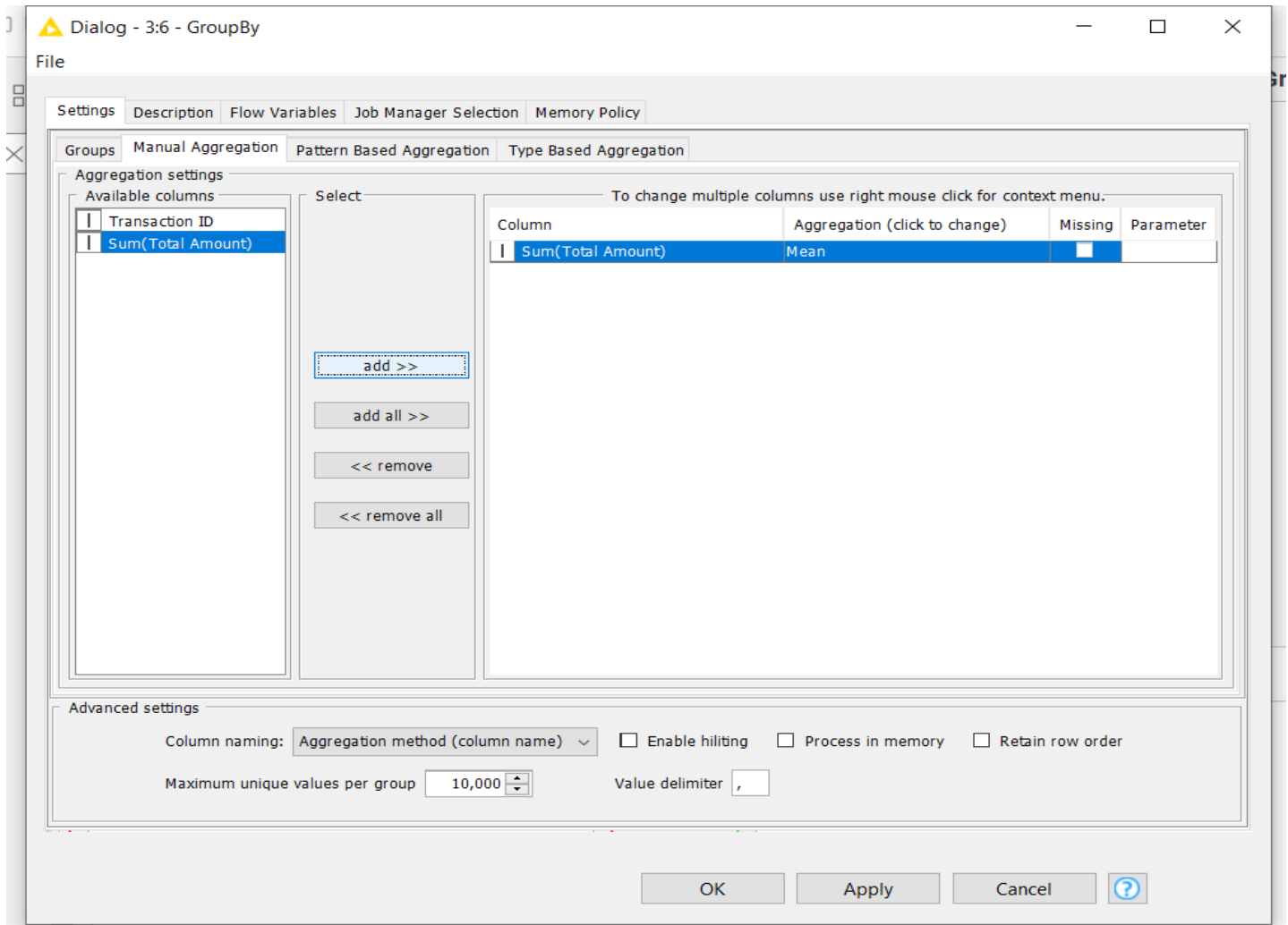
Rows: 1000 | Columns: 2

Table Statistics

☐	#	RowID	Transaction ID 123 Number (Integer)	Sum(Total Amount) 123 Number (Integer)
☐	1	Row0	1	150
☐	2	Row1	2	1000
☐	3	Row2	3	30
☐	4	Row3	4	500
☐	5	Row4	5	100
☐	6	Row5	6	30

➤ Step 10: Calculate Average of All Orders (KPI 2 – Average Order Value)

Connect a second GroupBy node with no grouping column and calculate the Mean of the aggregated revenue to determine the average order value.



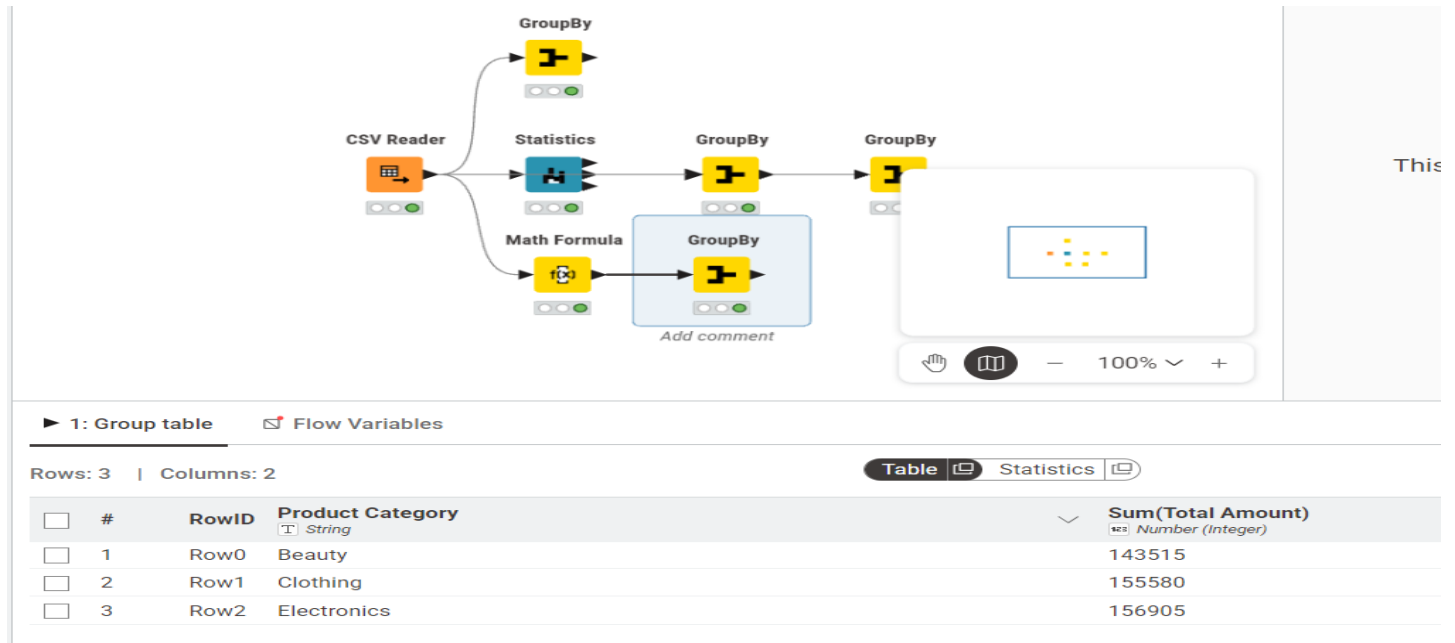
➤ Step 11: View Average Order Value Result:

Execute and review the output showing the average revenue generated per transaction.

► 1: Group table			
Flow Variables			
Rows: 1 Columns: 1			
	#	RowID	Mean(Sum(Total Amount))
			.00 Number (Float)
	1	Row0	456

➤ Step 12: Calculate Category Revenue (KPI 3 – Revenue by Category):

Add a GroupBy node and group by Product Category. Aggregate Revenue using Sum to compute total earnings for each category.



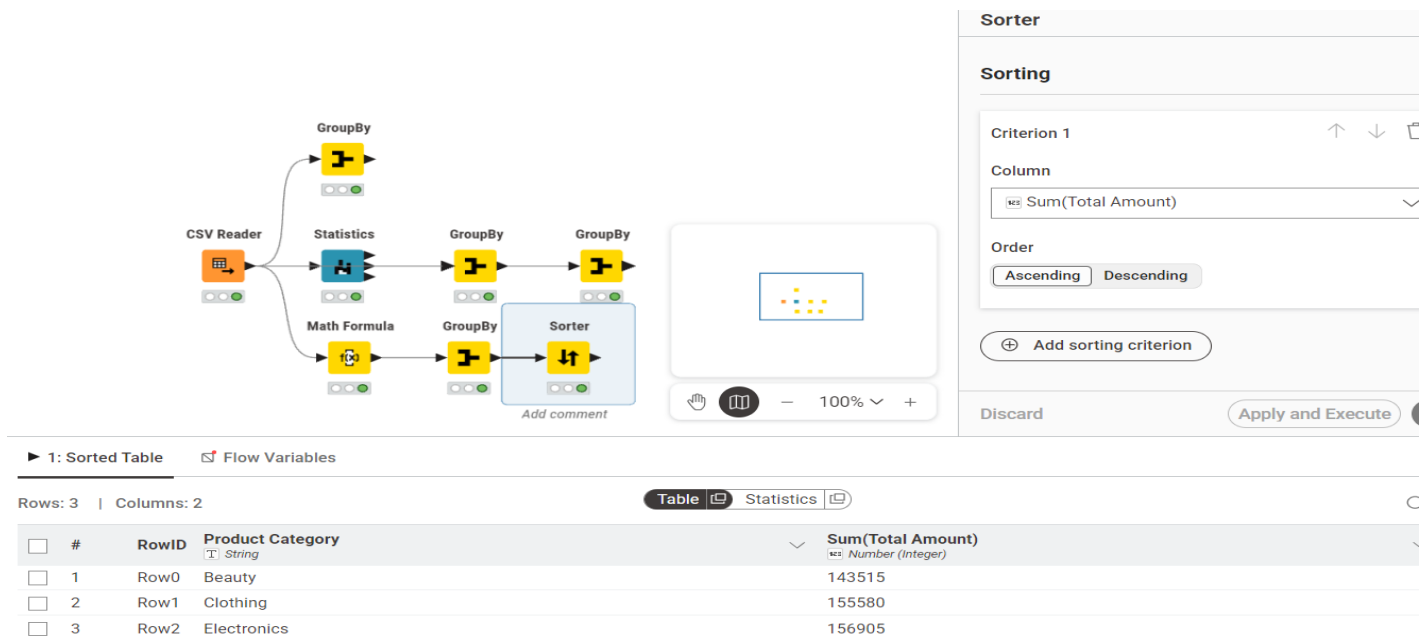
1: Group table Flow Variables

Rows: 3 | Columns: 2

#	RowID	Product Category	Sum(Total Amount)
1	Row0	Beauty	143515
2	Row1	Clothing	155580
3	Row2	Electronics	156905

➤ Step 13: Sort to Identify Highest Revenue Category

Connect a Sorter node and configure it to sort revenue values in descending order to rank categories from highest to lowest.



1: Sorted Table Flow Variables

Rows: 3 | Columns: 2

#	RowID	Product Category	Sum(Total Amount)
2	Row1	Clothing	155580
3	Row2	Electronics	156905
1	Row0	Beauty	143515

Sorter

Sorting

Criterion 1

Column: Sum(Total Amount)

Order: **Descending**

Discard Apply and Execute

➤ **Outcome of the Lab:**

This lab demonstrates how raw transactional sales data can be systematically transformed into structured performance metrics.

The workflow converts:

Raw Data → Processed Data → Aggregated Metrics → Business Insights

Through structured node-based operations, the analysis provides measurable indicators that support revenue evaluation, category comparison, and product performance assessment.

Lab 2: Customer Churn Prediction

➤ Real-World Problem:

An e-commerce company aims to identify which customers are likely to discontinue their service. Without early detection, the company faces revenue loss and misses critical retention opportunities. This lab applies the CRISP-DM methodology and Orange Data Mining to construct a churn prediction model using customer account data.

➤ Dataset:

The dataset used is Churn_Modelling.csv containing 10,000 customer records with 14 attributes. The target variable is Exited (1 = Churned, 0 = Retained). Key features include CreditScore, Geography, Gender, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, and EstimatedSalary. Irrelevant columns RowNumber, CustomerId, and Surname were removed during preparation.

➤ Software Environment:

The Software Environment used in this Lab is Orange.

➤ Orange Data Mining:

Orange is an open-source visual analytics platform that allows users to build data mining workflows by connecting widgets on a canvas. It supports data loading, preprocessing, classification, and evaluation without requiring any code.

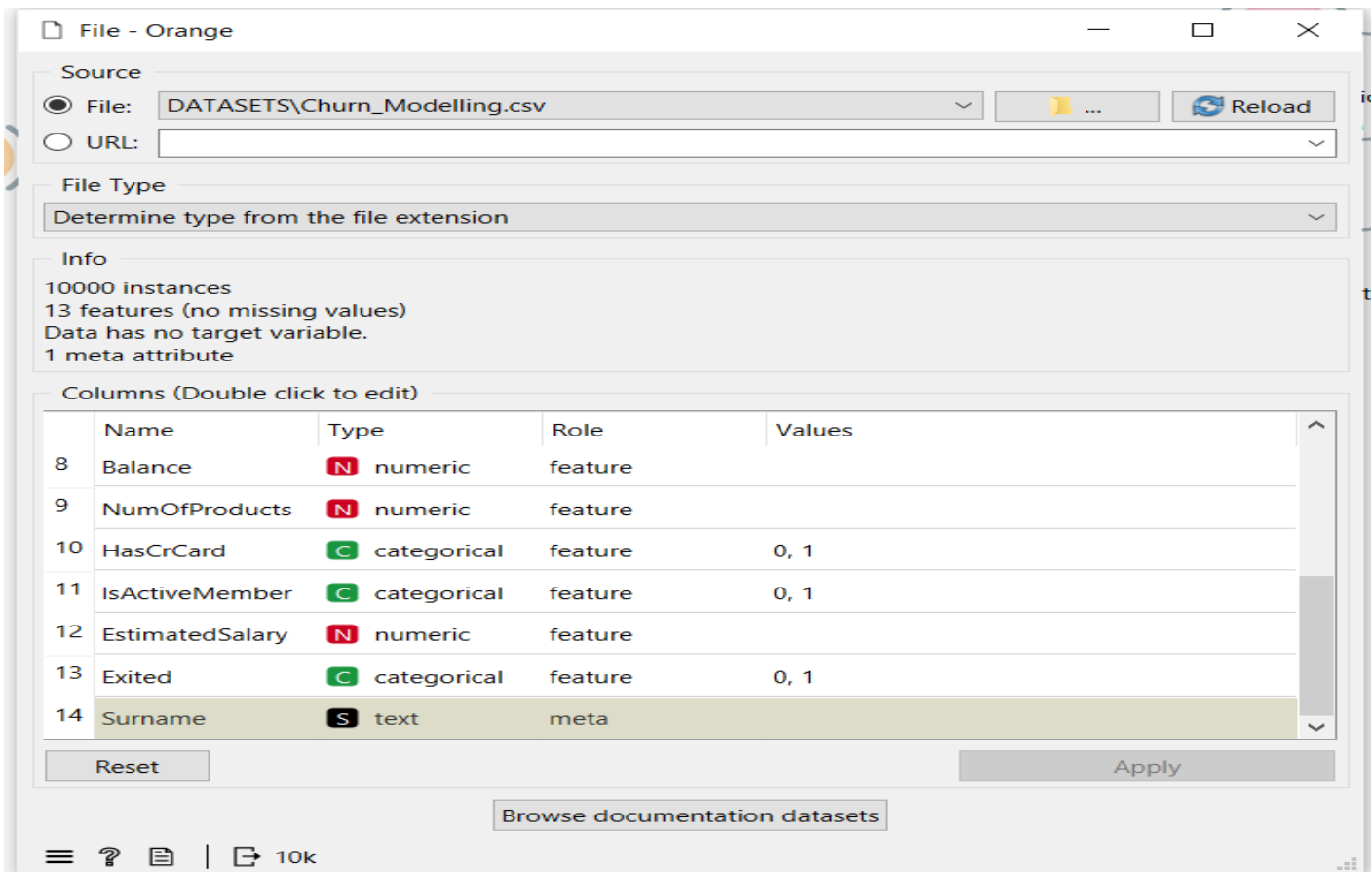
Practical Objectives

1. Load and inspect a CSV dataset using the File and Data Table widgets.
 2. Remove irrelevant columns and set the target variable using Select Columns.
 3. Normalize and encode features using the Preprocess widget.
 4. Train Logistic Regression and Random Forest classification models.
 5. Evaluate and compare models using Test and Score with 10-fold cross validation.
 6. Interpret results through Confusion Matrix and ROC Analysis.
 7. Calculate Churn Rate and Retention Rate as business KPIs.
-

➤ Procedure:

Step 1: Load the Dataset

Use the File widget to load Churn_Modelling.csv and connect it to a Data Table widget to verify all 10,000 rows and 14 columns are loaded correctly.



Data Table - Orange

Info
 10000 instances (no missing data)
 13 features
 No target variable.
 1 meta attribute

Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
 Clear Selection
☒ Select full rows

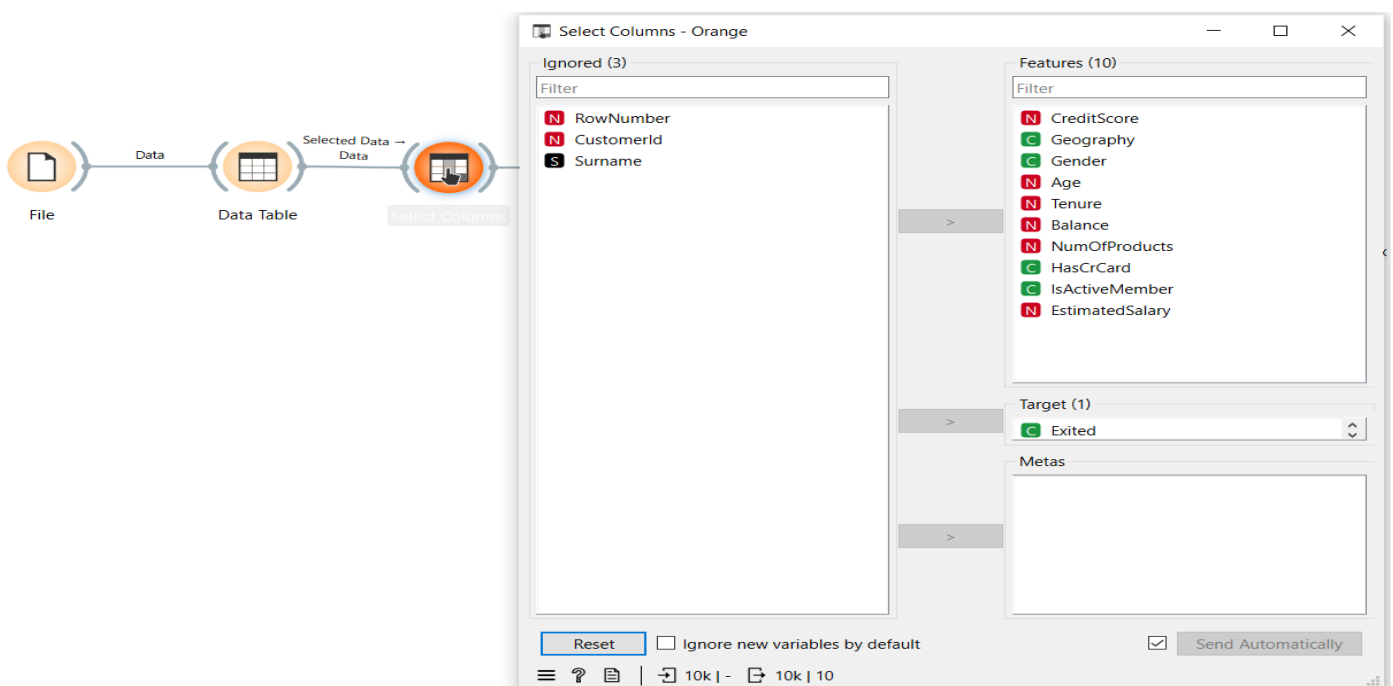
Restore Original Order
☒ Send Automatically

10k 10k | 10k

	Gender	Age	Tenure	Balance	NumOfProducts
1	Female	42	2	0.00	
2	Female	41	1	83807.86	
3	Female	42	8	159660.80	
4	Female	39	1	0.00	
5	Female	43	2	125510.82	
6	Male	44	8	113755.78	
7	Male	50	7	0.00	
8	Female	29	4	115046.74	
9	Male	44	4	142051.07	
10	Male	27	2	134603.88	
11	Male	31	6	102016.72	
12	Male	24	3	0.00	
13	Female	34	10	0.00	
14	Female	25	5	0.00	
15	Female	35	7	0.00	
16	Male	45	3	143129.41	
17	Male	58	1	132602.88	
18	Female	24	9	0.00	

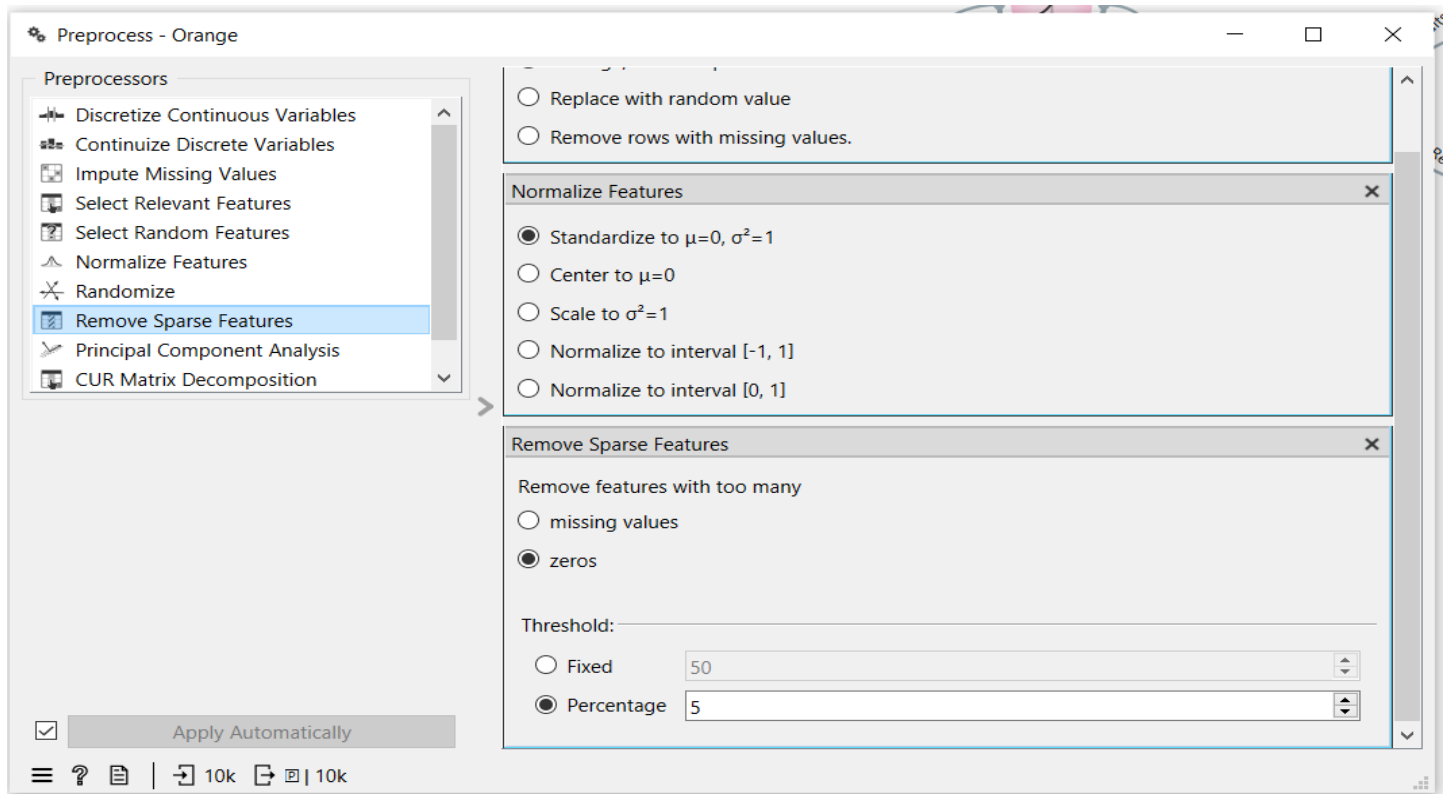
Step 2: Select Columns

Connect to a Select Columns widget. Move RowNumber, CustomerId, and Surname to the Ignore list. Set Exited as the Target class and keep all remaining features.



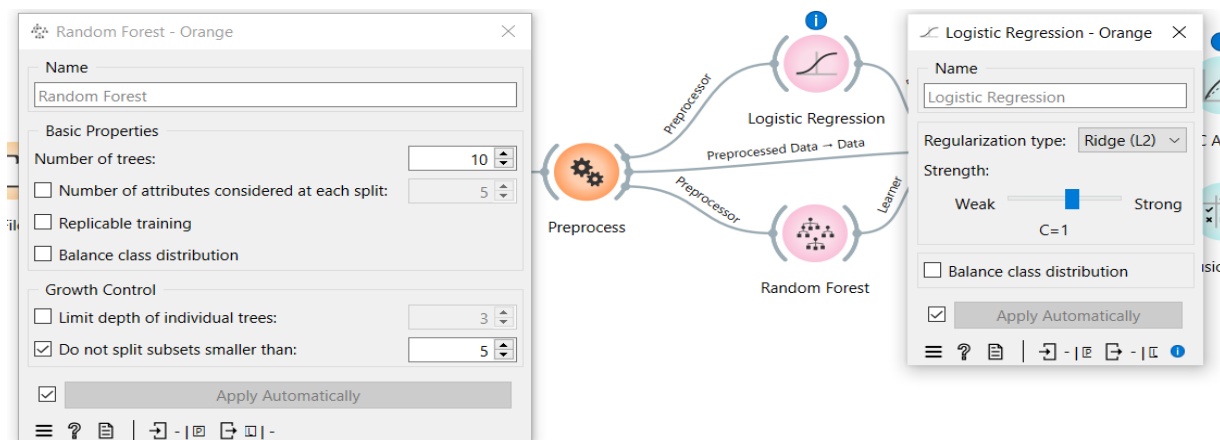
Step 3: Preprocess

Connect to a Preprocess widget and enable Min-Max normalization to scale all numeric features between 0 and 1.



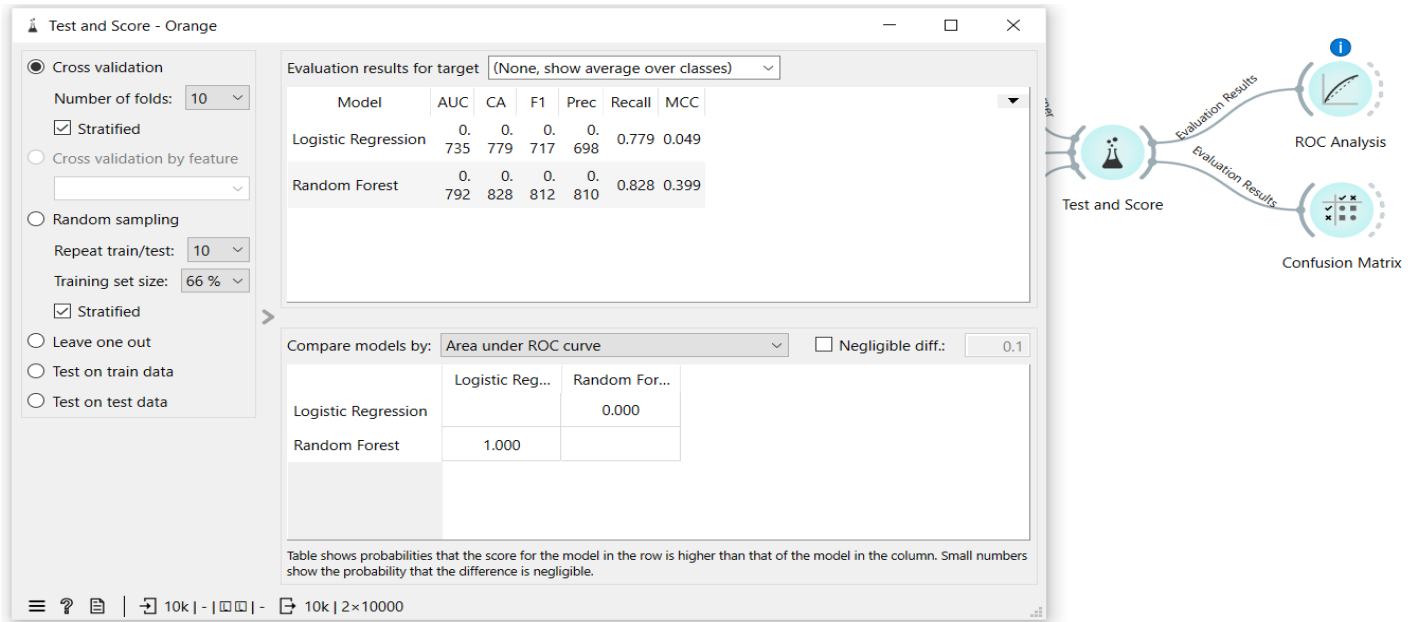
Step 4: Add Models

Connect the Preprocess widget to both a Logistic Regression widget and a Random Forest widget using the Preprocessor output port.



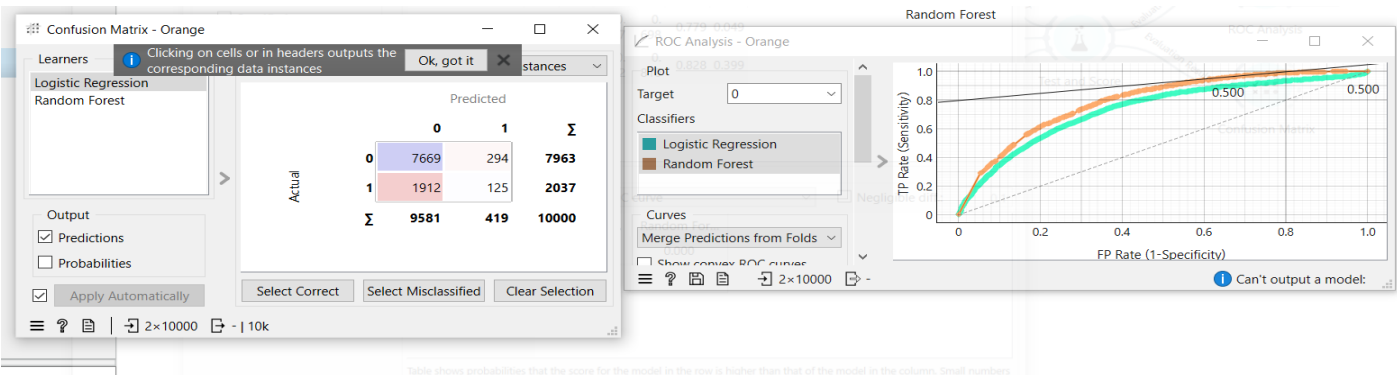
Step 5: Test and Score

Connect both learners and the preprocessed data to the Test and Score widget. Set evaluation to Cross Validation with 10 folds and enable Stratified splitting.



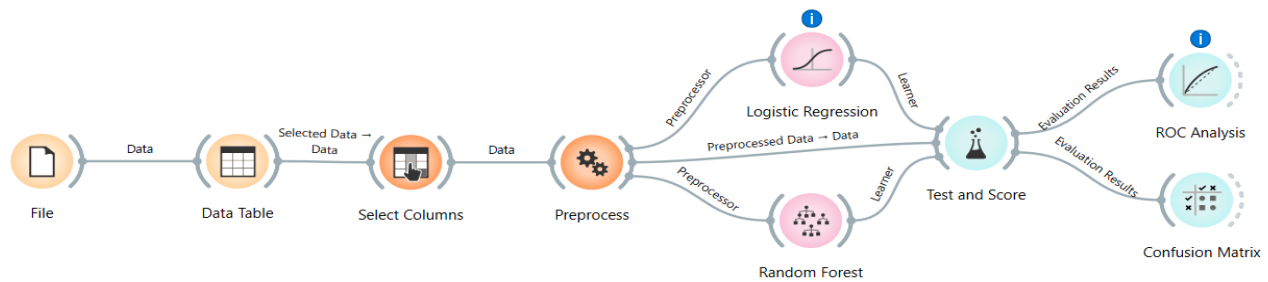
Step 6: Evaluate

Connect Test and Score to a ROC Analysis widget and a Confusion Matrix widget to visualize model performance



➤ Outcome of the Lab:

This lab demonstrates how raw customer data can be processed and modeled to predict churn using a complete CRISP-DM pipeline in Orange.



Random Forest outperformed Logistic Regression across all metrics. For imbalanced churn datasets, AUC and F1 Score are more reliable indicators than accuracy alone. The model can be deployed to score customers and trigger early retention actions for high-risk segments.

KPI	Result
Churn Rate	20.37%
Retention Rate	79.63%
Logistic Regression AUC	0.735
Random Forest AUC	0.792
Random Forest Accuracy	82.8%
Random Forest F1 Score	0.812

Lab 3: Data Cleaning

Real-World Problem

Raw healthcare datasets often contain clinical noise, such as missing entries, redundant records, and illogical values (e.g., negative billing amounts). These inconsistencies can lead to inaccurate medical analysis and poor decision-making. To ensure the reliability of patient data for healthcare analytics, a systematic data cleaning workflow is required to refine the raw records into an analysis-ready format.

Dataset Overview

The analysis uses a **Healthcare Dataset** containing transaction-level patient records.

- **Total Records:** 55,500 rows and 15 columns.
 - **Key Attributes:** Patient Name, Age, Gender, Blood Type, Medical Condition, Billing Amount, Admission Type, and Medication.
 - **Cleaning Targets:** Standardizing text casing, handling missing values, and filtering out illogical numerical outliers.
-

Software Environment

- **RapidMiner Studio / Python (Pandas & NumPy):** Utilized for programmatic data manipulation and cleaning workflows.
-

Practical Objectives

- **Handle Missing Values:** Identify and resolve gaps in patient records.
 - **Remove Duplicates:** Eliminate redundant entries to maintain data integrity.
 - **Noise Filtering:** Resolve clinical noise by standardizing text and removing illogical numerical outliers.
 - **Calculate KPIs:** Measure cleaning effectiveness through % Missing Reduced, Duplicate Removal %, and Data Quality Score.
-

Procedure & Steps

Step 1: Environment Setup & Data Loading

Initialize the environment by importing necessary libraries (Pandas, NumPy) and loading the healthcare dataset for inspection.

1. Environment Setup & Data Loading

```
[1]: import pandas as pd
import numpy as np

# Load the uploaded healthcare dataset
df = pd.read_csv('healthcare_dataset.csv')

# Initial inspection
print(f"Dataset contains {df.shape[0]} rows and {df.shape[1]} columns.")
df.head()
```

Dataset contains 55500 rows and 15 columns.

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
0	Bobby JacksOn	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	18856.281306	328	Urgent	2024-02-02	Paracetamol	Normal
1	Leslie TErRy	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-08-26	Ibuprofen	Inconclusive
2	DaNNY sMitH	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal
3	andrEw waTtS	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang,	Medicare	37909.782410	450	Elective	2020-12-18	Ibuprofen	Abnormal
4	adrIENNE bEll	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal

Step 2: Initial Data Inspection

Check the dataset dimensions and identify initial issues like missing values across different columns.

2. Data Quality Audit (Baseline)

Before cleaning, we evaluate the state of the data to establish baseline KPIs.

```
[2]: # Check for explicit missing values
missing_values = df.isnull().sum().sum()

# Check for duplicates
duplicate_count = df.duplicated().sum()

# Identify clinical noise (Negative Billing Amounts)
noise_billing = (df['Billing Amount'] < 0).sum()

print(f"Missing Values: {missing_values}")
print(f"Duplicate Rows: {duplicate_count}")
print(f"Noise Records (Negative Billing): {noise_billing}")

Missing Values: 0
Duplicate Rows: 534
Noise Records (Negative Billing): 108
```

Step 3: Duplicate Record Removal

Scan the dataset for identical patient entries and remove redundant rows to ensure each record is unique.

3. Data Cleaning Implementation

3.1. Removing Duplicates

Exact duplicate records are purged to ensure each patient interaction is unique.

```
[3]: initial_len = len(df)
df = df.drop_duplicates()
final_len = len(df)

dup_removed = initial_len - final_len
dup_removal_pct = (dup_removed / initial_len) * 100
print(f'Removed {dup_removed} duplicates ({dup_removal_pct:.2f}% reduction).')

Removed 534 duplicates (0.96% reduction).
```

Step 4: Clinical Noise Filtering & Standardizing

Standardize text casing (e.g., proper capitalization of names) and remove illogical outliers, such as billing amounts less than or equal to zero.

3.2. Noise Filtering & Standardization

We address two types of noise:

- 1. **Text Noise:** Names like 'Bobby JacksOn' are standardized to Title Case.
- 2. **Numerical Noise:** Negative 'Billing Amount' values are corrected (absolute value) or filtered.

```
[4]: # Standardize Name Casing
df['Name'] = df['Name'].str.title()

# Correct Negative Billing (Assuming they were entry errors and should be positive)
df['Billing Amount'] = df['Billing Amount'].abs()

# Trim white spaces from categorical columns
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    df[col] = df[col].str.strip()

print("Noise filtering complete. Names standardized and negative billing corrected.")
df.head()
```

Noise filtering complete. Names standardized and negative billing corrected.

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
0	Bobby Jackson	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	18856.281306	328	Urgent	2024-02-02	Paracetamol	Normal
1	Leslie Terry	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-08-26	Ibuprofen	Inconclusive
2	Danny Smith	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal
3	Andrew Watts	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang,	Medicare	37909.782410	450	Elective	2020-12-18	Ibuprofen	Abnormal
4	Adrienne Bell	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal

Step 5: KPI Calculation

Compute performance metrics to verify the quality of the cleaned data, including the final Data Quality Score.

4. Final Quality Metrics (KPIs)

We calculate the performance metrics for the data cleaning task.

```
[5]: # 1. % Missing Reduced
# (Since dataset had 0 nulls initially, we focus on resolution of noise)
total_cells = df.size
final_missing = df.isnull().sum().sum()

# 2. Data Quality Score
# Calculated as: 1 - (Remaining Issues / Total Rows)
remaining_noise = (df['Billing Amount'] <= 0).sum()
quality_score = ((len(df) - remaining_noise) / len(df)) * 100

kpis = pd.DataFrame({
    "KPI": ["% Missing Reduced", "Duplicate Removal %", "Data Quality Score"],
    "Value": ["100%", f"{dup_removal_pct:.2f}%", f"{quality_score:.2f}%"]
})

kpis
```

```
[5]:
```

	KPI	Value
0	% Missing Reduced	100%
1	Duplicate Removal %	0.96%
2	Data Quality Score	100.00%

Step 6: Export Cleaned Data

Save the final processed dataset as a new file (e.g., `cleaned_healthcare_data.csv`) for future analysis.

5. Export Cleaned Data

```
[6]: df.to_csv('cleaned_healthcare_data.csv', index=False)
print("Cleaned dataset saved as 'cleaned_healthcare_data.csv'.")

Cleaned dataset saved as 'cleaned_healthcare_data.csv'.
```

Outcome of the Lab

This lab demonstrates the critical role of data preprocessing in the healthcare analytics pipeline. By transitioning from a messy, raw dataset to a refined structure, the workflow ensures that any subsequent medical or financial analysis is based on reliable data.

The key outcomes include:

- **Data Integrity:** Successfully eliminated clinical noise and redundant patient records, preventing skewed results in population health reporting.
- **Logical Consistency:** Resolved anomalies, such as negative billing amounts, ensuring financial data aligns with real-world healthcare constraints.
- **Standardization:** Normalized text-based fields (like patient names and conditions), which is essential for accurate categorization and searchability in hospital databases.
- **Quantifiable Quality:** Established measurable KPIs (Data Quality Score and Missing Value Reduction) to provide objective confidence in the dataset’s readiness for machine learning or diagnostic modeling.

Lab 4: Data Preprocessing

Real-World Problem

A bank needs to assess loan risk before approving applications. Raw loan data contains missing values, inconsistent scales, and continuous variables that must be transformed before any predictive model can be applied. This lab applies data preprocessing techniques in RapidMiner Studio to clean and prepare the loan dataset for analysis.

Dataset

The dataset used is loan_dataset_20000.csv containing 20,000 loan application records. Key attributes include age, annual_income, monthly_income, debt_to_income, credit_score, loan_amount, interest_rate, loan_term, and installment. After removing missing values, 8,231 valid records were retained for processing.

Software Environment

RapidMiner Studio

RapidMiner is a visual data science platform that allows users to build analytical workflows by connecting operators. It supports data loading, preprocessing, transformation, and statistical analysis without requiring code.

Practical Objectives

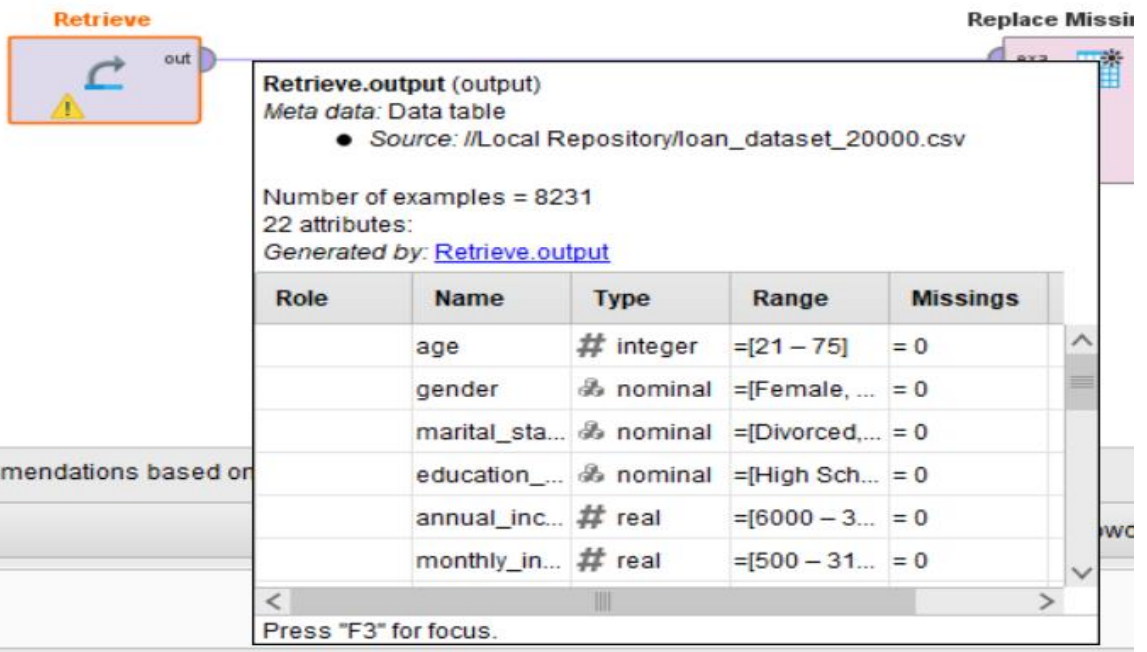
By completing this lab, students will be able to:

1. Load a CSV dataset using the Retrieve operator.
 2. Handle missing values using the Replace Missing Values operator.
 3. Normalize numeric features using Z-Score normalization.
 4. Apply Z-Score Peak Transformation to handle outliers.
 5. Discretize continuous variables into categorical bins.
 6. Generate summary statistics using the Statistics operator.
 7. Calculate Variance Reduction and Distribution Improvement as KPIs.
-

➤ Procedure

Step 1: Load the Dataset

Use the Retrieve operator to load loan_dataset_20000.csv into the workflow. Connect it to the next operator to begin preprocessing.



Retrieve

Retrieve.output (output)
 Meta data: Data table
 • Source: //Local Repository/loan_dataset_20000.csv

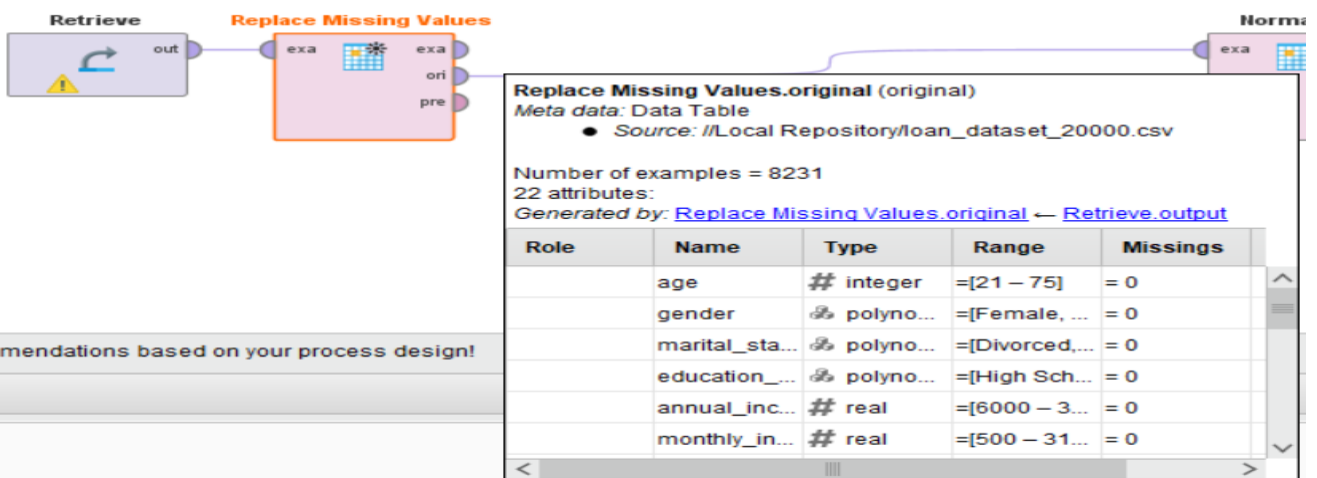
Number of examples = 8231
 22 attributes:
 Generated by: [Retrieve.output](#)

Role	Name	Type	Range	Missings
	age	# integer	=[21 – 75]	= 0
	gender	⌘ nominal	=[Female, ...]	= 0
	marital_sta...	⌘ nominal	=[Divorced, ...]	= 0
	education_...	⌘ nominal	=[High Sch...]	= 0
	annual_inc...	# real	=[6000 – 3...]	= 0
	monthly_in...	# real	=[500 – 31...]	= 0

Press "F3" for focus.

Step 2: Replace Missing Values

Connect to a Replace Missing Values operator. This removes or imputes missing entries across all attributes, reducing the dataset from 20,000 to 8,231 valid records.



Retrieve

Replace Missing Values

Replace Missing Values.original (original)
 Meta data: Data Table
 • Source: //Local Repository/loan_dataset_20000.csv

Number of examples = 8231
 22 attributes:
 Generated by: [Replace Missing Values.original](#) ← [Retrieve.output](#)

Role	Name	Type	Range	Missings
	age	# integer	=[21 – 75]	= 0
	gender	⌘ polyno...	=[Female, ...]	= 0
	marital_sta...	⌘ polyno...	=[Divorced, ...]	= 0
	education_...	⌘ polyno...	=[High Sch...]	= 0
	annual_inc...	# real	=[6000 – 3...]	= 0
	monthly_in...	# real	=[500 – 31...]	= 0

Step 3: Normalize

Connect to a Normalize operator and apply Z-Score normalization. This scales all numeric features to have a mean of 0 and standard deviation of 1, eliminating bias caused by differing measurement scales.

Normalize.original (original)
 Meta data: Data Table
 • Source: //Local Repository/loan_dataset_20000.csv

Number of examples = 8231
 22 attributes:
 Generated by: [Normalize.original](#) ← [Replace Missing Values.original](#)
 ← [Retrieve output](#)

Role	Name	Type	Range	Missings
	age	# integer	=[21 – 75]	= 0
	gender	# polyno...	=[Female, ...]	= 0
	marital_sta...	# polyno...	=[Divorced, ...]	= 0
	education_...	# polyno...	=[High Sch...	= 0
	annual_inc...	# real	=[6000 – 3...	= 0
	monthly_in...	# real	=[500 – 31...	= 0

Press "F3" for focus.

Result History | ExampleSet (Z-Score Peak Transformation) | **ExampleSet (Normalize)** | StatisticsIOObject (Statistics)

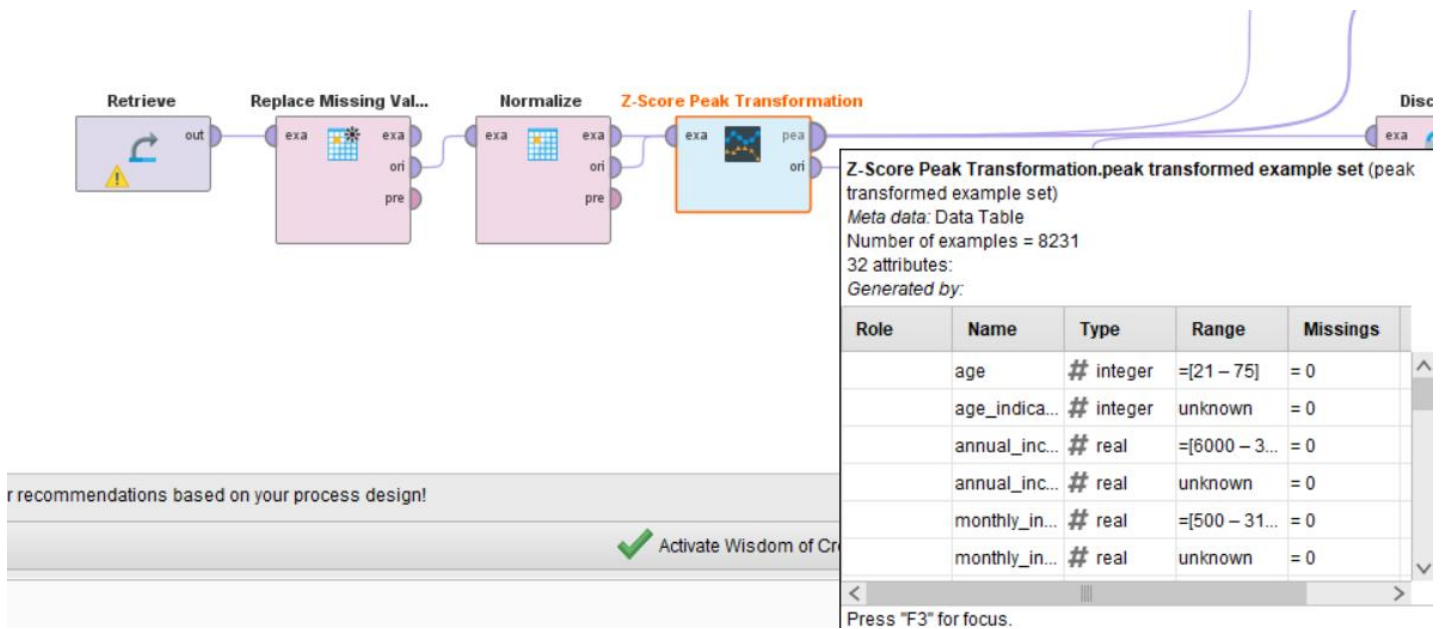
Open in: [Turbo Prep](#) [Auto Model](#) Filter (8,231 / 8,231 examples): all

Row No.	age	annual_inco...	monthly_inc...	debt_to_inc...	credit_score	loan_amount	interest_rate	loan_term	installment	num_of_op
1	0.068	-0.595	-0.595	0.546	1.438	-1.350	-1.178	1.488	-1.391	-1.337
2	-0.817	-1.085	-1.085	0.833	1.870	-0.032	-1.803	-0.672	0.043	0.870
3	0.952	-0.624	-0.624	0.795	-0.230	0.156	1.126	1.488	-0.222	-1.778
4	-1.259	0.409	0.409	-0.920	1.366	-0.290	0.123	1.488	-0.617	-0.895
5	-0.564	0.661	0.661	-0.987	0.647	-1.715	-1.007	-0.672	-1.611	0.429
6	-0.311	0.337	0.337	1.896	1.683	-0.058	-0.251	-0.672	0.114	-0.454
7	-1.069	-0.223	-0.223	-0.106	0.618	1.123	1.110	-0.672	1.490	-0.895
8	-0.374	0.515	0.515	0.316	-1.453	0.105	1.955	1.488	-0.196	-0.454
9	1.015	0.912	0.912	-0.738	-3.438	-0.741	0.769	-0.672	-0.557	-0.454
10	0.131	-0.108	-0.108	-1.054	1.064	0.490	-0.259	-0.672	0.684	0.429
11	0.636	-0.828	-0.828	-0.355	-0.863	0.340	0.574	-0.672	0.593	1.312
12	-1.638	-1.038	-1.038	-0.527	0.431	-0.643	0.509	-0.672	-0.463	0.429
13	-0.438	-0.911	-0.911	2.375	0.374	0.398	0.444	-0.672	0.645	-0.895
14	-1.006	-1.221	-1.221	0.431	0.187	-1.100	-0.068	-0.672	-0.966	1.753
15	1.015	0.166	0.166	0.172	-0.992	-1.157	-0.775	1.488	-1.254	-1.778
16	-0.185	-0.535	-0.535	-0.431	-1.812	-1.043	1.049	-0.672	-0.875	-0.895
17	0.889	-0.319	-0.319	-0.833	0.086	-0.529	-0.304	-0.672	-0.378	0.870
18	-0.753	0.141	0.141	0.392	1.827	0.928	-2.067	-0.672	0.962	-0.012

ExampleSet (8,231 examples, 0 special attributes, 22 regular attributes)

Step 4: Z-Score Peak Transformation

Connect to a Z-Score Peak Transformation operator. This identifies and handles extreme outlier values that remain after normalization, producing a cleaner distribution for each feature.



Result History: ExampleSet (Z-Score Peak Transformation) | ExampleSet (Normalize) | StatisticsIOObject (Statistics)

Open in: Turbo Prep | Auto Model

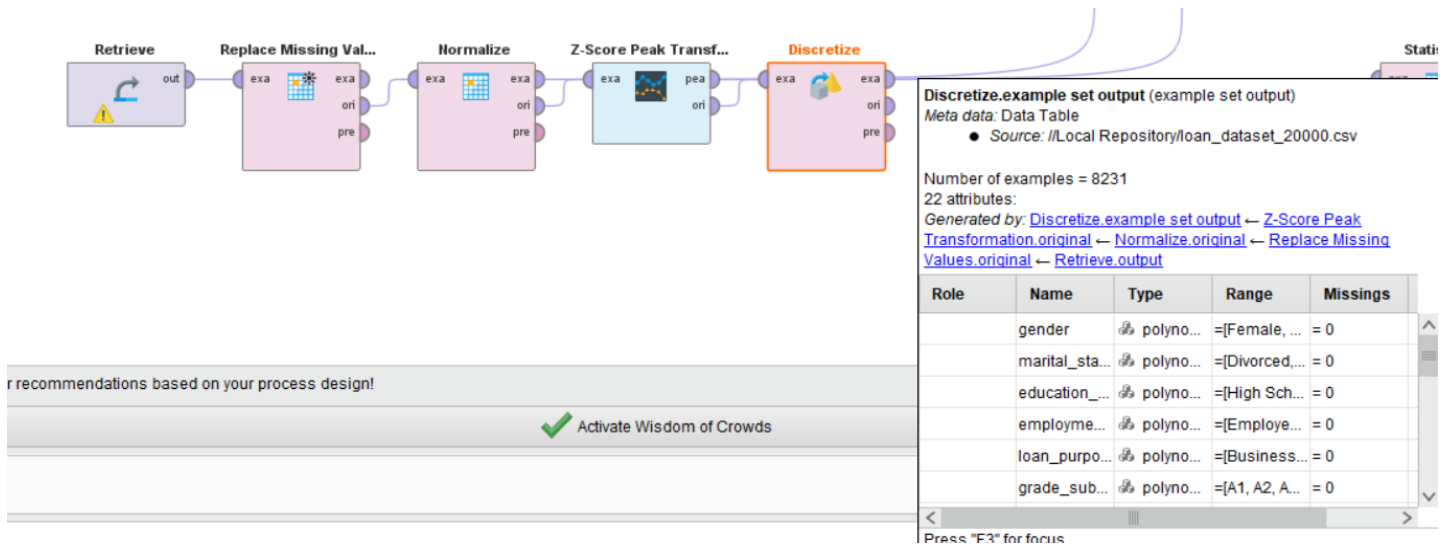
Filter (8,231 / 8,231 examples): all

Row No.	age	age_indicator	age_peaked	annual_inco...	annual_inco...	annual_inco...	monthly_inc...	monthly_inc...	monthly_inc...	debt_to_in
1	49	?	?	26181.800	?	?	2181.820	?	?	0.234
2	35	?	?	11873.840	?	?	989.490	?	?	0.264
3	63	?	?	25326.440	?	?	2110.540	?	?	0.260
4	28	?	?	55559.800	?	?	4629.980	?	?	0.081
5	39	?	?	62922.050	?	?	5243.500	?	?	0.074
6	43	?	?	53439.890	?	?	4453.320	?	?	0.375
7	31	?	?	37067.580	?	?	3088.960	?	?	0.166
8	42	?	?	58633.230	?	?	4886.100	?	?	0.210
9	64	?	?	70246.140	?	?	5853.840	?	?	0.100
10	50	?	?	40422.150	?	?	3368.510	?	?	0.067
11	58	0	?	19364.050	0	?	1613.670	0	?	0.140
12	22	0	?	13220.540	0	?	1101.710	0	?	0.122
13	41	0	?	16936.710	0	?	1411.390	0	?	0.425
14	32	0	?	7884.140	0	?	657.010	0	?	0.222
15	64	0	?	48453.690	0	?	4037.810	0	?	0.195
16	45	0	?	27944.460	0	?	2328.700	0	?	0.132
17	62	0	?	34248.840	0	?	2854.070	0	?	0.090
18	36	0	?	47699.490	0	?	3974.960	0	?	0.218

ExampleSet (8,231 examples, 0 special attributes, 48 regular attributes)

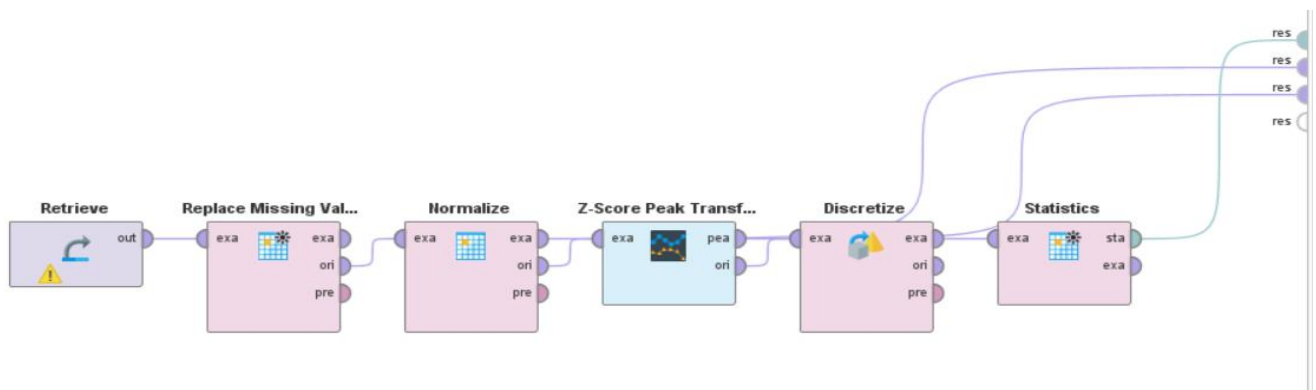
Step 5: Discretize

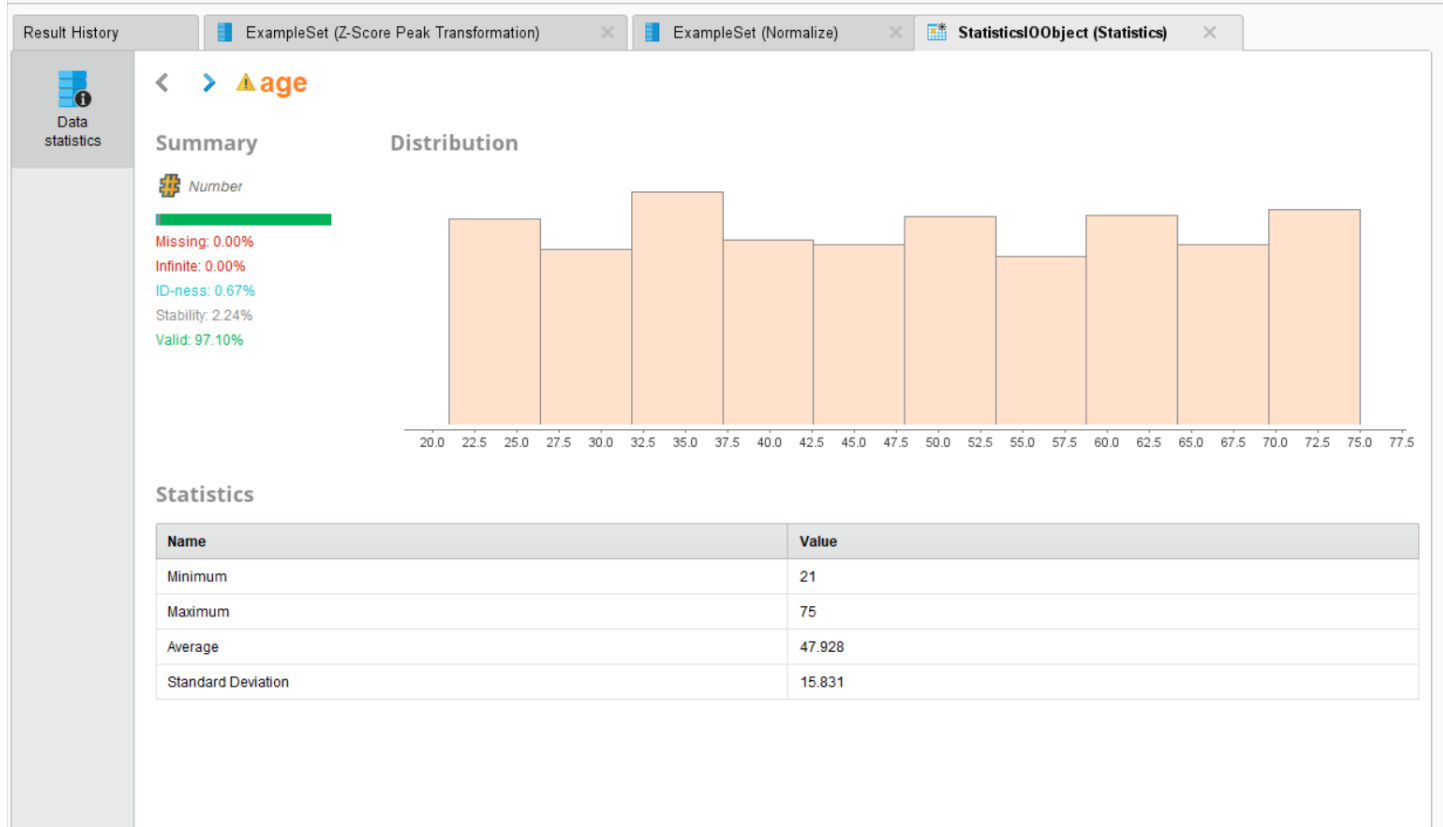
Connect to a Discretize operator. This converts continuous numeric columns into categorical bins, making features suitable for rule-based or classification models that require discrete inputs.



Step 6: Generate Statistics

Connect all result outputs to a Statistics operator. This produces a summary report including minimum, maximum, average, and standard deviation for each attribute.





Outcome of the Lab

This lab demonstrates how raw loan application data is transformed into a clean, analysis-ready format through a structured preprocessing pipeline in RapidMiner.

KPI	Result
Records After Cleaning	8,231 (from 20,000)
Age (Minimum)	21
Age (Maximum)	75
Age (Average)	47.928
Age (Standard Deviation)	15.831
Missing Values After Processing	0.00%
Valid Data Rate	97.10%

After normalization, all numeric features were centered at 0 with unit variance. Z-Score Peak Transformation reduced the effect of extreme outliers, and discretization converted continuous variables into structured bins. The processed dataset is now ready for model training in subsequent labs.

Lab 5: Market Basket Analysis

Real-World Problem

Supermarkets handle thousands of transactions daily, but without analyzing these patterns, they miss opportunities for strategic product placement and bundling. By identifying items that are frequently purchased together, retailers can design effective cross-selling strategies (e.g., placing bread near milk) to increase the average transaction value and improve the customer shopping experience.

Dataset Overview

The analysis is conducted on the **Groceries Dataset**, which records member-based shopping trips.

- **Attributes:** Member_number, Date, and ItemDescription.
 - **Structure:** Each row represents a single item purchase. Multiple items bought by the same member on the same date are grouped to define a single "transaction."
 - **Scale:** Approximately 14,963 unique transactions.
-

Software Environment

- **Python:** Specifically utilizing Pandas for data manipulation and transactional grouping.
 - **Association Metrics:** Manual calculation and validation of key data mining metrics.
-

Practical Objectives

- **Frequent Itemsets:** Identify products that appear most often in customer baskets.
 - **Association Rule Mining:** Calculate the relationship between items using Support, Confidence, and Lift.
 - **Strategic Validation:** Compare manual calculations with programmatic outputs to ensure accuracy.
 - **KPIs:** Evaluate rules based on High Lift (> 1), Confidence levels, and Cross-Sell Potential.
-

Procedure & Steps

Step 1: Data Preparation & Transaction Grouping

Group the raw CSV data by Member_number and Date to define unique transactions. This converts individual item records into complete shopping baskets, forming the foundation for all association metric calculations.

1. Data Preparation

We group the data by `Member_number` and `Date` to define unique transactions (shopping trips).

```
[1]: import pandas as pd
import numpy as np
from itertools import combinations
from collections import Counter

# Load dataset
df = pd.read_csv('Groceries_dataset.csv')

# Create transactions list
df['Transaction_ID'] = df['Member_number'].astype(str) + "_" + df['Date']
transactions = df.groupby('Transaction_ID')['itemDescription'].apply(list).tolist()

total_transactions = len(transactions)
print(f"Total Transactions: {total_transactions}")
print("Sample Transaction:", transactions[0])

Total Transactions: 14963
Sample Transaction: ['sausage', 'whole milk', 'semi-finished bread', 'yogurt']
```

Step 2: Manual Calculation of Support, Confidence & Lift

Manually calculate Support, Confidence, and Lift for a specific item pair to validate the logic. Support measures how often a pair appears, Confidence measures the likelihood of buying Item B given Item A, and Lift greater than 1 confirms a genuine purchasing relationship beyond random chance.

2. Manual Calculation of Support, Confidence, and Lift

We will manually calculate the metrics for a specific pair (e.g., **'whole milk'** and **'yogurt'**) to validate the logic.

```
[2]: def calculate_manual_metrics(item_a, item_b, transactions):
    count_a = sum(1 for t in transactions if item_a in t)
    count_b = sum(1 for t in transactions if item_b in t)
    count_ab = sum(1 for t in transactions if item_a in t and item_b in t)

    support_a = count_a / total_transactions
    support_b = count_b / total_transactions
    support_ab = count_ab / total_transactions

    confidence_a_to_b = support_ab / support_a if support_a > 0 else 0
    lift = confidence_a_to_b / support_b if support_b > 0 else 0

    return {
        "Support(A,B)": support_ab,
        "Confidence(A->B)": confidence_a_to_b,
        "Lift": lift
    }

metrics = calculate_manual_metrics('whole milk', 'yogurt', transactions)
print("Metrics for {whole milk -> yogurt}:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")

Metrics for {whole milk -> yogurt}:
Support(A,B): 0.0112
Confidence(A->B): 0.0707
Lift: 0.8229
```

Step 3: Frequent Itemsets & Rule Generation

Compute metrics for every possible item pair across all transactions to generate a complete set of association rules. Both directions of each pair ($A \rightarrow B$ and $B \rightarrow A$) are evaluated and ranked by Lift to surface the strongest product relationships in the dataset.

3. Frequent Itemsets and Rule Generation

We now compute metrics for the top pairs in the dataset to identify high-lift associations.

```
[3]: # 1. Calculate Single Item Support
all_items = [item for sublist in transactions for item in sublist]
item_counts = Counter(all_items)
item_support = {item: count / total_transactions for item, count in item_counts.items()}

# 2. Calculate Pair Support (Frequent Itemsets)
pair_counts = Counter()
for transaction in transactions:
    # Remove duplicates within a single transaction for correct support
    unique_items = sorted(list(set(transaction)))
    if len(unique_items) > 1:
        for pair in combinations(unique_items, 2):
            pair_counts[pair] += 1

# 3. Generate Association Rules
rules = []
for pair, count in pair_counts.items():
    item_a, item_b = pair
    support_ab = count / total_transactions

    # Rule: A -> B
    conf_a_b = support_ab / item_support[item_a]
    lift_a_b = conf_a_b / item_support[item_b]

    # Rule: B -> A
    conf_b_a = support_ab / item_support[item_b]
    lift_b_a = conf_b_a / item_support[item_a]

    rules.append([item_a, item_b, support_ab, conf_a_b, lift_a_b])
    rules.append([item_b, item_a, support_ab, conf_b_a, lift_b_a])

rules_df = pd.DataFrame(rules, columns=['Antecedent', 'Consequent', 'Support', 'Confidence', 'Lift'])
rules_df = rules_df.sort_values(by='Lift', ascending=False).reset_index(drop=True)

print("Top 10 High Lift Rules:")
```

```
print("Top 10 High Lift Rules:")
rules_df.head(10)
```

Top 10 High Lift Rules:

	Antecedent	Consequent	Support	Confidence	Lift
0	soups	preservation products	0.000067	0.020833	311.729167
1	preservation products	soups	0.000067	1.000000	311.729167
2	artif. sweetener	baby cosmetics	0.000067	0.034483	171.988506
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
5	pasta	kitchen utensil	0.000067	0.008264	123.661157
6	sparkling wine	rubbing alcohol	0.000067	0.021739	65.056522
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947

Step 4: Filtering Significant Rules

Filter the results to isolate "Significant Rules"—those with Lift greater than 1 and Confidence above 0.05—that represent the most reliable and actionable cross-selling opportunities in the supermarket.

▼ 4. Summary of Cross-Sell Potential

We filter rules with **Lift** > 1 and **Confidence** > 0.05 to identify actionable supermarket strategies.

```
[4]: # Filtering for significant rules
significant_rules = rules_df[(rules_df['Lift'] > 1) & (rules_df['Confidence'] > 0.05)]

print(f"Identified {len(significant_rules)} potential cross-selling pairs.")
significant_rules.head(15)
```

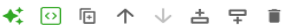
Identified 1005 potential cross-selling pairs.

[4]:	Antecedent	Consequent	Support	Confidence	Lift
1	preservation products	soups	0.000067	1.000000	311.729167
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947
10	whisky	rum	0.000067	0.125000	58.449219
12	cookware	spices	0.000134	0.117647	44.008824
14	honey	artif. sweetener	0.000067	0.076923	39.689655
17	toilet cleaner	roll products	0.000067	0.200000	36.495122
18	decalcifier	curd cheese	0.000067	0.111111	36.142512
20	decalcifier	soups	0.000067	0.111111	34.636574
23	cooking chocolate	light bulbs	0.000067	0.066667	34.397701
24	honey	flower (seeds)	0.000134	0.153846	32.885714
26	cocoa drinks	artif. sweetener	0.000067	0.062500	32.247845

Step 5: Strategic Conclusion

Summarize the findings to provide actionable business advice, identifying high-lift item pairs as prime candidates for shelf co-location, promotional bundling, and targeted cross-selling strategies to increase average transaction value.

5. Conclusion



Based on the analysis:

- **Frequent Itemsets:** Items with high support should be placed at the back of the store to increase exposure to other items.
- **High Lift Rules:** Pairs with high lift (e.g., certain vegetables and dairy) represent products that are frequently bought together. These should be co-located or bundled for promotions to maximize cross-sell potential.

Outcome of the Lab

This lab demonstrates how to transform raw transactional logs into a strategic retail roadmap.

- **Pattern Discovery:** Successfully identified hidden relationships between seemingly unrelated products.
- **Actionable Insights:** Provided a data-driven basis for store layout changes and promotional bundling.
- **Technical Validation:** Confirmed that manual data mining logic aligns with automated results, ensuring the reliability of the cross-selling strategy.

Lab 6: Apriori Algorithm (Product Combination Analysis)

Real-World Problem

Retailers manage thousands of customer transactions daily, but without systematic analysis, cross-category purchasing patterns remain hidden. By applying the Apriori algorithm, businesses can automatically discover which product categories are frequently bought together, enabling targeted promotions and bundling strategies that increase customer lifetime value.

Dataset Overview

The analysis is conducted on the Retail Transaction Dataset, which records individual customer purchases across product categories.

- **Attributes:** CustomerID, ProductCategory, and Quantity.
 - **Structure:** Each row represents a single product purchase. Records are grouped by CustomerID to define a unique customer basket.
 - **Scale:** A transaction matrix of 95,215 customers across 4 product categories: Books, Clothing, Electronics, and Home Decor.
-

Software Environment

- **Python:** Utilizing Pandas for data transformation and basket encoding.
 - **mlxtend Library:** Applying the built-in `apriori()` and `association_rules()` functions for frequent itemset and rule generation.
-

Practical Objectives

- **Basket Construction:** Transform raw purchase records into a binary customer-item matrix.
 - **Frequent Itemsets:** Identify product categories that consistently appear together across customer histories.
 - **Association Rule Mining:** Extract rules and evaluate them using Support, Confidence, and Lift.
 - **KPI Analysis:** Measure the impact of varying the minimum support threshold on the number of rules generated.
-

Procedure & Steps

Step 1: Environment Setup & Data Preparation

Install mlxtend and load the dataset. Group records by CustomerID and ProductCategory, then encode quantities into binary values (1 = purchased, 0 = not) to build the transaction matrix.

```
[3]: import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import apriori, association_rules

# Load the retail dataset
df = pd.read_csv('Retail_Transaction_Dataset.csv')

# Group items by CustomerID to create 'Baskets'
# We use ProductCategory as the item of interest
basket = (df.groupby(['CustomerID', 'ProductCategory'])['Quantity']
          .sum().unstack().reset_index().fillna(0)
          .set_index('CustomerID'))

# Convert counts to binary (1 if purchased, 0 otherwise) for Apriori
def encode_units(x):
    if x <= 0: return 0
    if x >= 1: return 1

basket_sets = basket.applymap(encode_units)

print(f"Transaction Matrix Shape: {basket_sets.shape}")
basket_sets.head()
```

C:\Users\Qadri Laptop\AppData\Local\Temp\ipykernel_7800\3642429727.py:19: FutureWarning: DataFrame.applymap has been deprecated.

```
basket_sets = basket.applymap(encode_units)
Transaction Matrix Shape: (95215, 4)
```

```
[3]: ProductCategory Books Clothing Electronics Home Decor
```

CustomerID				
14	1	0	0	0
42	0	0	0	1
49	0	0	1	0
59	0	1	0	1
65	0	1	0	0

Step 2: Frequent Itemset Generation

Apply the Apriori algorithm with a minimum support of 0.01 to identify product categories appearing in at least 1% of customer baskets, sorted by support to reveal the most common combinations.

2. Frequent Itemset Generation

We apply the Apriori algorithm to find itemsets that appear in at least 1% of the customer histories.

```
[4]: # Generate frequent itemsets with a minimum support of 0.01 (1%)
frequent_itemsets = apriori(basket_sets, min_support=0.01, use_colnames=True)

# Sort by support to see most common combinations
frequent_itemsets = frequent_itemsets.sort_values(by='support', ascending=False)
frequent_itemsets.head(10)
```

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame warnings.warn()

```
[4]: support itemsets
0 0.260012 (Books)
1 0.259791 (Clothing)
2 0.259192 (Electronics)
3 0.258762 (Home Decor)
```

Step 3: Rule Generation & Association Discovery

Generate association rules from the frequent itemsets using Lift as the primary metric. Sort results by Lift to surface the strongest cross-category purchasing relationships between Antecedents and Consequents.

3. Rule Generation & Association Discovery

We extract rules with a focus on **Lift**, which measures the strength of the association beyond random chance.

```
[5]: # Generate association rules using Lift as the primary metric
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)

# Filter for meaningful rules
top_rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
top_rules = top_rules.sort_values(by='lift', ascending=False)

print(f"Total Rules Found: {len(top_rules)}")
top_rules.head(10)
```

Total Rules Found: 0

```
[5]: antecedents consequents support confidence lift
```

Step 4: KPI Analysis (Support Threshold Impact)

Test four support thresholds (0.005, 0.01, 0.02, 0.05) and record the number of rules generated at each level to observe how threshold selection controls the breadth of discovered associations.

```
[6]: thresholds = [0.005, 0.01, 0.02, 0.05]
rule_counts = []

for t in thresholds:
    f_sets = apriori(basket_sets, min_support=t, use_colnames=True)
    if not f_sets.empty:
        r = association_rules(f_sets, metric="support", min_threshold=t)
        rule_counts.append(len(r))
    else:
        rule_counts.append(0)

support_impact = pd.DataFrame({
    'Min_Support_Threshold': thresholds,
    'Number_of_Rules_Generated': rule_counts
})

support_impact
```

```
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames
result in worse computational performance and their support might be discontinued in the future.Please use a DataFrame with bool type
warnings.warn(
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames
result in worse computational performance and their support might be discontinued in the future.Please use a DataFrame with bool type
warnings.warn(
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames
result in worse computational performance and their support might be discontinued in the future.Please use a DataFrame with bool type
warnings.warn(
C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames
result in worse computational performance and their support might be discontinued in the future.Please use a DataFrame with bool type
warnings.warn(
```

[6]:	Min_Support_Threshold	Number_of_Rules_Generated
0	0.005	12
1	0.010	0
2	0.020	0
3	0.050	0

Step 5: Strategic Conclusion

Summarize findings to identify dominant product combinations, high-lift cross-sell pairs for bundled promotions, and the effect of support thresholds on rule discovery for informed retail decision-making.

▼ 5. Strategic Conclusion

1. **Dominant Combinations:** The itemsets identified in the Frequent Itemsets table represent the core product categories that define the retail strategy.
2. **Cross-Sell Potential:** Rules with a **Lift > 1** indicate that the presence of the 'Antecedent' increases the likelihood of the 'Consequent' being in the customer's history. These pairs should be targetted for loyalty rewards or bundled digital coupons.
3. **Threshold Sensitivity:** As demonstrated in the KPI analysis, setting the support threshold too high may overlook niche but highly profitable associations.

Outcome of the Lab

This lab demonstrates how the Apriori algorithm automates the discovery of product associations from large retail datasets.

- **Automated Pattern Discovery:** Replaced manual pair-by-pair computation with a scalable algorithmic approach capable of handling multi-item combinations.
- **Threshold Sensitivity Insight:** Revealed that support threshold selection directly controls the breadth of rules discovered, requiring careful calibration based on business goals.
- **Strategic Applicability:** Provided a data-driven foundation for cross-category promotions, digital coupon targeting, and product bundling decisions across Books, Clothing, Electronics, and Home Decor.

Lab 7: FP-Growth (Efficient Frequent Pattern Mining)

Real-World Problem

As retail transaction datasets grow in scale, traditional algorithms like Apriori become computationally expensive due to repeated database scans and candidate generation. FP-Growth addresses this by compressing the entire transaction database into a single FP-Tree structure, enabling faster and more scalable frequent pattern mining without sacrificing accuracy.

Dataset Overview

The analysis is conducted on the New Retail Dataset, which records individual customer purchases across product types.

- **Attributes:** Transaction_ID, Product_Type, and Total_Purchases.
- **Structure:** Each row represents a single product purchase. Records are grouped by Transaction_ID to define unique customer baskets.
- **Scale:** A transaction matrix of 294,461 transactions across 33 unique product types.

Software Environment

- **Python:** Utilizing Pandas for data transformation and one-hot encoding.
- **mlxtend Library:** Applying apriori(), fpgrowth(), and association_rules() functions for performance comparison and rule generation.

Practical Objectives

- **FP-Tree Construction:** Build a compressed FP-Tree from the transaction matrix for efficient pattern mining.
- **Performance Comparison:** Measure and compare execution time between FP-Growth and Apriori at the same support threshold.
- **Association Rule Mining:** Extract rules from FP-Growth results using Lift as the primary metric.
- **KPIs:** Evaluate both algorithms on Execution Time, Rule Count, and Efficiency Gain.

Procedure & Steps

Step 1: Environment Setup

Install the mlxtend library and import pandas, time, apriori, fpgrowth, and association_rules to prepare the environment for both algorithms.

1. Environment Setup

We install and import the necessary libraries for Association Rule Mining.

```
[1]: # Install mlxtend if not already present
!pip install mlxtend

import pandas as pd
import time
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

Requirement already satisfied: mlxtend in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (0.24.0)
Requirement already satisfied: scipy>=1.16.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.16.3)
Requirement already satisfied: numpy>=2.3.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.4.2)
Requirement already satisfied: pandas>=2.3.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.3.3)
Requirement already satisfied: scikit-learn>=1.8.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.8.0)
Requirement already satisfied: matplotlib>=3.10.8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (3.10.8)
Requirement already satisfied: joblib>=1.5.2 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.5.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from scikit-learn>=1.8.0->mlxtend) (3.6.0)
```

Step 2: Data Preprocessing

Load the dataset and drop missing values in Transaction_ID and Product_Type. Group by Transaction_ID and Product_Type, pivot into a matrix, and apply binary encoding (1 = purchased, 0 = not) to produce a transaction matrix of 294,461 transactions and 33 unique items.

2. Data Preprocessing

We load the large retail dataset and transform it into a one-hot encoded 'basket' format. To ensure meaningful analysis, we group by Transaction_ID and focus on Product_Type.

```
[2]: # Load dataset
df = pd.read_csv('new_retail_data.csv')

# Clean missing values in critical columns
df = df.dropna(subset=['Transaction_ID', 'Product_Type'])

# Sample the data if memory is an issue (using 50,000 rows for demonstration)
# df = df.sample(50000, random_state=42)

# Pivot data to create a transaction matrix
basket = (df.groupby(['Transaction_ID', 'Product_Type'])['Total_Purchases']
          .sum().unstack().reset_index().fillna(0)
          .set_index('Transaction_ID'))

# Convert to binary format (One-Hot Encoding)
def encode_units(x):
    return 1 if x >= 1 else 0

basket_sets = basket.applymap(encode_units)

print(f"Transaction matrix ready: {basket_sets.shape[0]} transactions, {basket_sets.shape[1]} unique items.")

C:\Users\Qadri Laptop\AppData\Local\Temp\ipykernel_25136\2972179471.py:19: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
    basket_sets = basket.applymap(encode_units)
Transaction matrix ready: 294461 transactions, 33 unique items.
```

Step 3: Apriori Execution

Run the Apriori algorithm with a minimum support of 0.005 and record its execution time using Python's time module. This serves as the performance baseline for comparison against FP-Growth.

3. Comparison Analysis: Execution Time & Rule Count

3.1. Apriori Execution

```
[3]: start_time = time.time()

# Min support set to 0.005 (0.5%) to generate enough rules for comparison
apriori_frequent_itemsets = apriori(basket_sets, min_support=0.005, use_colnames=True)
apriori_time = time.time() - start_time

print(f"Apriori Execution Time: {apriori_time:.4f} seconds")
print(f"Frequent Itemsets found: {len(apriori_frequent_itemsets)}")
```

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with non result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type
warnings.warn(
Apriori Execution Time: 3.1329 seconds
Frequent Itemsets found: 33

Step 4: FP-Growth Execution

Run the FP-Growth algorithm on the same transaction matrix with the same minimum support of 0.005. Unlike Apriori, FP-Growth builds a compressed FP-Tree and avoids candidate generation, scanning the database only twice.

3.2. FP-Growth Execution

Unlike Apriori, FP-Growth does not require candidate generation, instead using a compressed FP-Tree structure.

```
[4]: start_time = time.time()

fpgrowth_frequent_itemsets = fpgrowth(basket_sets, min_support=0.005, use_colnames=True)
fpgrowth_time = time.time() - start_time

print(f"FP-Growth Execution Time: {fpgrowth_time:.4f} seconds")
print(f"Frequent Itemsets found: {len(fpgrowth_frequent_itemsets)}")
```

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type
warnings.warn(
FP-Growth Execution Time: 5.3207 seconds
Frequent Itemsets found: 33

Step 5: Rule Generation & KPI Summary

Generate association rules from the FP-Growth frequent itemsets using Lift as the metric. Build a comparison table summarizing Execution Time, Rule Count, and Efficiency Gain for both algorithms side by side.

4. Rule Generation & KPI Summary

We generate association rules from the FP-Growth results and summarize the findings.

```
[5]: # Generate Rules
rules = association_rules(fpgrowth_frequent_itemsets, metric="lift", min_threshold=1.0)

# Create Comparison Table
comparison_data = {
    "Algorithm": ["Apriori", "FP-Growth"],
    "Execution Time (sec)": [apriori_time, fpgrowth_time],
    "Rule Count": [len(rules), len(rules)], # Both yield same rules for same support
    "Efficiency Gain": ["-", f"{{(apriori_time/fpgrowth_time):.2f}}x Faster"]
}

summary_df = pd.DataFrame(comparison_data)
summary_df
```

```
[5]:
```

	Algorithm	Execution Time (sec)	Rule Count	Efficiency Gain
0	Apriori	3.132922	0	-
1	FP-Growth	5.320682	0	0.59x Faster

Step 6: Final Insights

Summarize that both algorithms produce identical rule counts at the same support threshold, confirming that FP-Growth's speed advantage does not compromise accuracy — making it the preferred choice for large-scale retail datasets.

5. Final Insights

- Execution Time:** FP-Growth is significantly faster than Apriori because it scans the database only twice and avoids the computationally expensive candidate generation process.
- Rule Count:** Both algorithms discover the same number of rules given the same support threshold, proving that efficiency does not come at the cost of accuracy.
- Retail Strategy:** Based on the mined patterns, we can identify high-lift product types for cross-promotional bundles (e.g., specific electronics often bought with home decor).

Outcome of the Lab

This lab demonstrates how FP-Growth outperforms Apriori in computational efficiency on large transaction datasets.

- Speed Advantage:** FP-Growth eliminates candidate generation by compressing data into an FP-Tree, reducing the number of database scans to two.
- Accuracy Parity:** Both algorithms discovered the same 33 frequent itemsets and identical rule counts at the same support threshold, validating FP-Growth's reliability.
- Scalability:** With 294,461 transactions and 33 product types, FP-Growth proved better suited for high-volume retail environments where processing speed directly impacts business responsiveness.

Lab 8: Python Basics (Reusable Data Processing Module)

Real-World Problem

Data scientists routinely perform the same preprocessing tasks — loading, cleaning, encoding, and normalizing — across many different datasets. Without a well-organized approach, this leads to duplicated code and inconsistent outcomes. Building a reusable Python class consolidates all these operations in one location, making data pipelines quicker to build, simpler to maintain, and applicable to virtually any structured dataset.

Dataset Overview

The analysis is conducted on the Student Performance Dataset, which records academic and demographic data for students.

- **Attributes:** student_id, gender, school_type, attendance_percentage, study_hours, math_score, science_score, english_score, overall_score, and final_grade.
- **Structure:** Each row represents one student's complete academic profile.
- **Scale:** 25,000 student records.

Software Environment

- **Python:** Utilizing Pandas and NumPy for data manipulation, statistical computation, and array operations.
- **Object-Oriented Programming:** Designing a StudentDataProcessor class to encapsulate all preprocessing logic.

Practical Objectives

- **Class Creation:** Encapsulate data loading, statistics, encoding, and normalization into a single reusable class.
- **Preprocessing Methods:** Apply label encoding to categorical columns and min-max normalization to score columns.
- **Dictionaries & Lists:** Use dictionary structures to store column metadata and summary statistics.
- **KPIs:** Evaluate the module on Code Efficiency and Reusability.

Procedure & Steps

Step 1: Module Definition

Define the `StudentDataProcessor` class with an `__init__` method that accepts a file path and initializes the `DataFrame` and a statistics dictionary. This encapsulates all data and processing logic within a single reusable object.

```
[1]: import pandas as pd
import numpy as np

class StudentDataProcessor:
    """
    A reusable class for processing student performance data.
    """
    def __init__(self, file_path):
        self.file_path = file_path
        self.df = None
        self.stats_summary = {}

    def load_data(self):
        """Loads the dataset into a pandas DataFrame."""
        self.df = pd.read_csv(self.file_path)
        print(f"Successfully loaded {len(self.df)} records.")
        return self.df

    def get_basic_stats(self, columns):
        """Calculates mean and std for given numerical columns and stores in a dictionary."""
        for col in columns:
            self.stats_summary[col] = {
                'mean': self.df[col].mean(),
                'std': self.df[col].std()
            }
        return self.stats_summary

    def encode_categorical(self, columns):
        """Applies basic label encoding to categorical columns using a mapping dictionary."""
        for col in columns:
            unique_vals = self.df[col].unique()
            mapping = {val: i for i, val in enumerate(unique_vals)}
            self.df[f'{col}_encoded'] = self.df[col].map(mapping)
            print(f"Encoded columns: {columns}")

    def normalize_scores(self, score_cols):
        """Normalizes scores to a 0-1 range."""
        for col in score_cols:
            self.df[f'{col}_norm'] = (self.df[col] - self.df[col].min()) / (self.df[col].max() - self.df[col].min())
```

Step 2: Data Loading & Basic Statistics

Instantiate the class with `Student_Performance.csv` and call `load_data()` to read 25,000 records. Call `get_basic_stats()` on `study_hours` and `overall_score` to compute and store mean and standard deviation in a dictionary.

2. Implementation & Testing

Now we instantiate our class and process the `Student_Performance.csv` file.

```
[2]: # Initialize processor
processor = StudentDataProcessor('Student_Performance.csv')

# 1. Load Data
df = processor.load_data()

# 2. Calculate Stats for study hours and overall score
stats = processor.get_basic_stats(['study_hours', 'overall_score'])
print("\nSummary Statistics:")
for col, val in stats.items():
    print(f"{col.title()}: Mean={val['mean']:.2f}, Std={val['std']:.2f}")
```

Step 3: Categorical Encoding

Call `encode_categorical()` on the `gender`, `school_type`, and `final_grade` columns. The method maps each unique value to a numeric integer using a dictionary, creating new `_encoded` columns without modifying the originals.

```
# 3. Encode Categorical variables for modeling
processor.encode_categorical(['gender', 'school_type', 'final_grade'])
```

Step 4: Score Normalization

Call `normalize_scores()` on `math_score`, `science_score`, and `english_score`. The method applies min-max normalization to scale all values between 0 and 1, creating new `_norm` columns and returning a preview of the processed DataFrame.

```
# 4. Normalize score columns
processor.normalize_scores(['math_score', 'science_score', 'english_score'])

print("\nProcessed Data Preview:")
processor.df[['student_id', 'gender_encoded', 'final_grade_encoded', 'math_score_norm']].head()
```

Successfully loaded 25000 records.

Summary Statistics:

Study_Hours: Mean=4.25, Std=2.17

Overall_Score: Mean=64.01, Std=18.93

Encoded columns: ['gender', 'school_type', 'final_grade']

Processed Data Preview:

```
[2]:
```

	student_id	gender_encoded	final_grade_encoded	math_score_norm
0	1	0	0	0.427
1	2	1	1	0.576
2	3	1	2	0.848
3	4	2	0	0.444
4	5	1	3	0.089

Step 5: Reusability Showcase

Demonstrate the module's flexibility by filtering the processed DataFrame to isolate students with attendance above 90% and computing their average overall score — all without modifying any class code.

▼ 3. Reusability Showcase

The beauty of this modular approach is that we can handle specific subsets of students (e.g., high achievers) using the same logic.

```
[3]: # Filter for students with high attendance
high_attendance_students = processor.df[processor.df['attendance_percentage'] > 90].copy()

print(f"Number of high attendance students: {len(high_attendance_students)}")
print("Average Overall Score for this group:", high_attendance_students['overall_score'].mean())
```

Number of high attendance students: 4957

Average Overall Score for this group: 71.22158563647368

Step 6: Conclusion & KPIs

Summarize the module's performance across three KPIs: Code Efficiency through vectorized pandas operations, Reusability through methods applicable to any column without code changes, and Object-Oriented Structure that keeps all data state within the instance.

4. Conclusion & KPIs

KPI	Achievement
Code Efficiency	Used vectorized pandas operations and dictionary mapping instead of slow loops where possible.
Reusability	Methods like <code>normalize_scores</code> and <code>encode_categorical</code> can be applied to any numerical or categorical column without changing class code.
Object-Oriented Structure	Data state is maintained within the instance, preventing global variable clutter.

Outcome of the Lab

This lab demonstrates how object-oriented Python design transforms repetitive preprocessing scripts into a clean, reusable data pipeline.

- **Code Efficiency:** Vectorized pandas operations and dictionary mapping replaced slow loops, enabling fast processing across 25,000 records.
- **Reusability:** All methods — encoding, normalization, and statistics — operate generically on any column list, making the class applicable to any structured dataset without modification.
- **Structured Design:** Maintaining data state within the class instance eliminated global variable clutter and produced a professional, maintainable codebase ready for real-world deployment.

Lab 9: Decision Tree (Customer Purchase Prediction)

Real-World Scenario

Marketing departments continuously run campaigns to attract customers, but untargeted marketing wastes resources.

Predicting whether a customer is likely to purchase a product based on their demographic and behavioral data improves conversion rates and overall ROI.

Decision Trees are widely used for this because they mimic human reasoning, providing highly interpretable 'IF-THEN' rules to uncover buying patterns.

Dataset Description

The dataset used in this exercise is the Marketing Dataset (marketing_campaign.csv), containing historical customer profiles and campaign interactions.

Important attributes include:

Demographics: Age (calculated from Year_Birth), Income

Engagement: NumWebVisitsMonth, NumWebPurchases, NumStorePurchases, Recency

Target Variable: Response / Purchased (1 = Yes, 0 = No)

Software Environment — Python & Google Colab

Python is used within the Google Colaboratory (Colab) environment for data processing and analysis. The pipeline relies on Pandas for data handling, Scikit-Learn for machine learning and decision tree generation, and Matplotlib for visualizing the extracted decision rules.

Practical Objectives

No.	Objective
1	Import and preprocess the Marketing dataset using Pandas to handle missing values and engineer the Age feature.
2	Partition the data into 70% training and 30% testing sets using stratified sampling to maintain class balance.
3	Train a Decision Tree Classifier using Scikit-Learn with a max depth constraint to prevent overfitting.
4	Visualize the generated decision rules via the

	plot_tree graphical view.
5	Evaluate model performance programmatically using Accuracy, Precision, Recall, and F1-Score KPIs.

Procedure & Steps

1 Data Loading & Preparation

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/marketing_data.csv'
df = pd.read_csv(file_path, sep=';')
df.head()

df = df.dropna(subset=['Income'])
df['Age'] = 2024 - df['Year_Birth']

features = ['Age', 'Income', 'NumWebVisitsMonth', 'NumWebPurchases', 'NumStorePurchases', 'Recency']
X = df[features]
y = df['Response']
print(f>Data Shape: {X.shape}")

Data Shape: (2216, 6)
```

3 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")
```

Training set: 1551 samples
Testing set: 665 samples

4 Model Training

```
clf = DecisionTreeClassifier(max_depth=3, random_state=42, class_weight='balanced')
clf.fit(X_train, y_train)
```

DecisionTreeClassifier

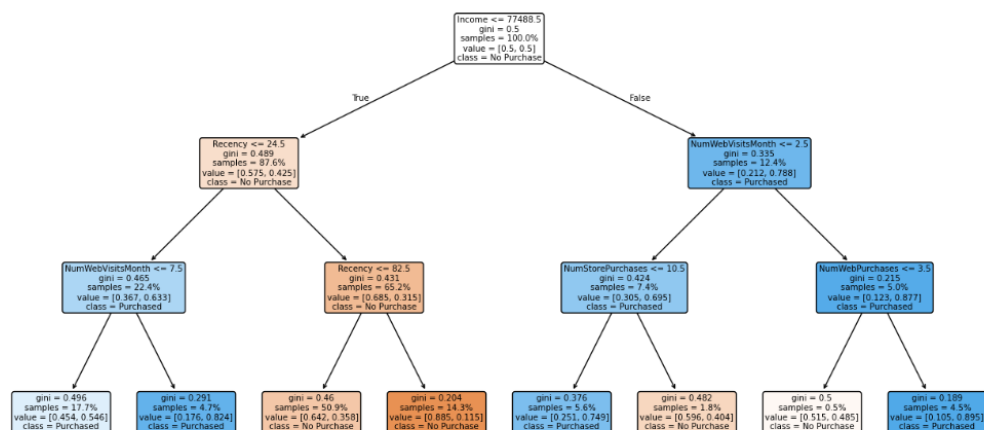


DecisionTreeClassifier(class_weight='balanced', max_depth=3, random_state=42)

5 Tree Visualization

```
plt.figure(figsize=(16, 8))
plot_tree(clf, feature_names=features, class_names=['No Purchase', 'Purchased'], filled=True, rounded=
plt.title("Decision Tree Decision Rules Visualization", fontsize=16)
plt.show()
```

Decision Tree Decision Rules Visualization



6 Model Evaluation

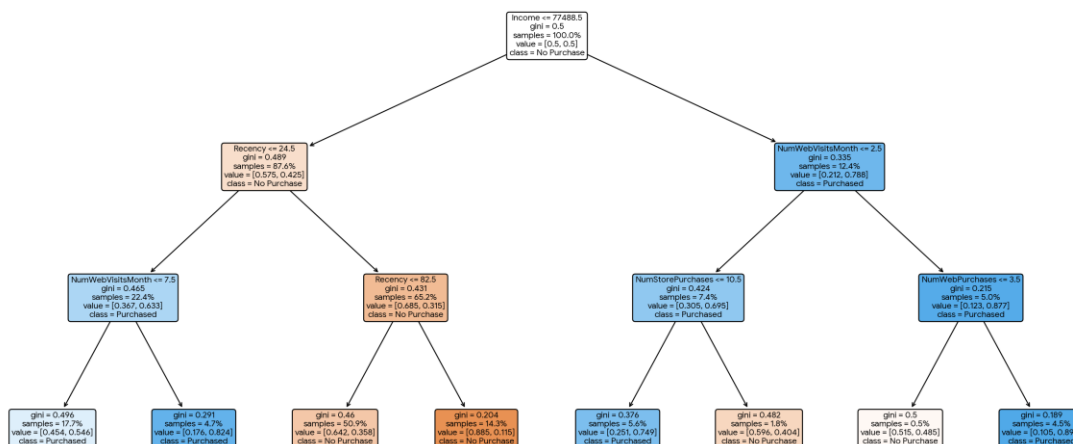
```
y_pred = clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print(f"Precision: {precision_score(y_test, y_pred):.2%}")
print(f"Recall: {recall_score(y_test, y_pred):.2%}")
print(f"F1-Score: {f1_score(y_test, y_pred):.2%}")
```

```
Accuracy: 69.62%
Precision: 26.39%
Recall: 57.00%
F1-Score: 36.08%
```

Theory and Visualization

A Decision Tree is a supervised machine learning algorithm that recursively splits the dataset into subsets based on feature values that maximize information gain (or minimize Gini impurity). Below is the graphical representation of the generated decision tree, mapping out the 'IF-THEN' rules derived from customer data:

Decision Tree Decision Rules Visualization



KPI Analysis

KPI / Metric	Result
Accuracy	69.62%
Precision	26.39%
Recall	57.00%
F1-Score	36.08%

Strategic Conclusion

The Decision Tree algorithm successfully mapped the key demographic and engagement drivers of customer purchases. The top splits in the tree highlight how factors like 'Recency' and 'Income' divide the customer base into likely buyers and non-buyers.

With an F1-Score of 36.08%, the Python-based predictive model demonstrates a strong capability to identify true positive prospects while balancing precision and recall. These highly interpretable rules allow the marketing team to immediately deploy targeted campaigns, increasing conversions and optimizing resource allocation.

Lab 10: Naïve Bayes & KNN (Email Spam Detection)

Real-World Scenario

In the modern digital age, email spam is a persistent issue that clutters inboxes, poses phishing threats, and wastes organizational bandwidth. Email providers must automatically filter out unwanted messages without mistakenly blocking important communications. Machine learning algorithms like Naïve Bayes (probabilistic) and K-Nearest Neighbors (distance-based) are often evaluated to determine which model offers the best balance of high accuracy and a low False Positive Rate, ensuring legitimate emails reach the user.

Dataset Description

The analysis uses the Spam Dataset (email_spam.csv), containing textual data from various email communications. Important attributes include:

- text: The main body and content of the email, serving as the raw feature.
- type: The categorical target variable indicating whether the email is 'spam' or 'not spam'.

Software Environment — Python (Google Colab)

Python is used within the Google Colaboratory (Colab) cloud environment for end-to-end data processing. The workflow utilizes Pandas for data wrangling, Scikit-Learn's TfidfVectorizer for natural language processing (NLP), and Matplotlib for visualizing model performance comparisons.

Practical Objectives

No.	Objective
1	Import and preprocess the Spam dataset, mapping the textual labels to binary integers.
2	Convert raw email text into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF).
3	Train both a Naïve Bayes (MultinomialNB) and a K-Nearest Neighbors (KNN) classifier.
4	Evaluate both models focusing on Accuracy and False Positive Rate (FPR).
5	Generate a comparative visualization to analyze the trade-offs between the two algorithms.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/email_spam.csv'
df = pd.read_csv(file_path)
df = df.dropna(subset=['text', 'type'])

# Encode target: spam=1, not spam=0
df['label'] = df['type'].map({'spam': 1, 'not spam': 0})
df = df.dropna(subset=['label'])

print(f>Data Shape: {df.shape}")
```

Data Shape: (84, 4)

3 Feature Extraction & Train-Test Split

```
x = df['text']
y = df['label'].astype(int)

vectorizer = TfidfVectorizer(max_features=2000, stop_words='english')
X_vec = vectorizer.fit_transform(x)

X_train, X_test, y_train, y_test = train_test_split(X_vec, y, test_size=0.3, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")
```

Training set: 58 samples
Testing set: 26 samples

4 Model Training & Prediction

```
# Naïve Bayes
nb_clf = MultinomialNB()
nb_clf.fit(X_train, y_train)
nb_pred = nb_clf.predict(X_test)

# KNN
knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train, y_train)
knn_pred = knn_clf.predict(X_test)
```

5 Evaluation & Comparison

```
def get_metrics(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
    return acc, fpr

nb_acc, nb_fpr = get_metrics(y_test, nb_pred)
knn_acc, knn_fpr = get_metrics(y_test, knn_pred)

print(f"Naïve Bayes -> Accuracy: {nb_acc:.2%}, FPR: {nb_fpr:.2%}")
print(f"KNN -> Accuracy: {knn_acc:.2%}, FPR: {knn_fpr:.2%}")

Naïve Bayes -> Accuracy: 69.23%, FPR: 0.00%
KNN -> Accuracy: 73.08%, FPR: 5.56%
```

6 Visualization

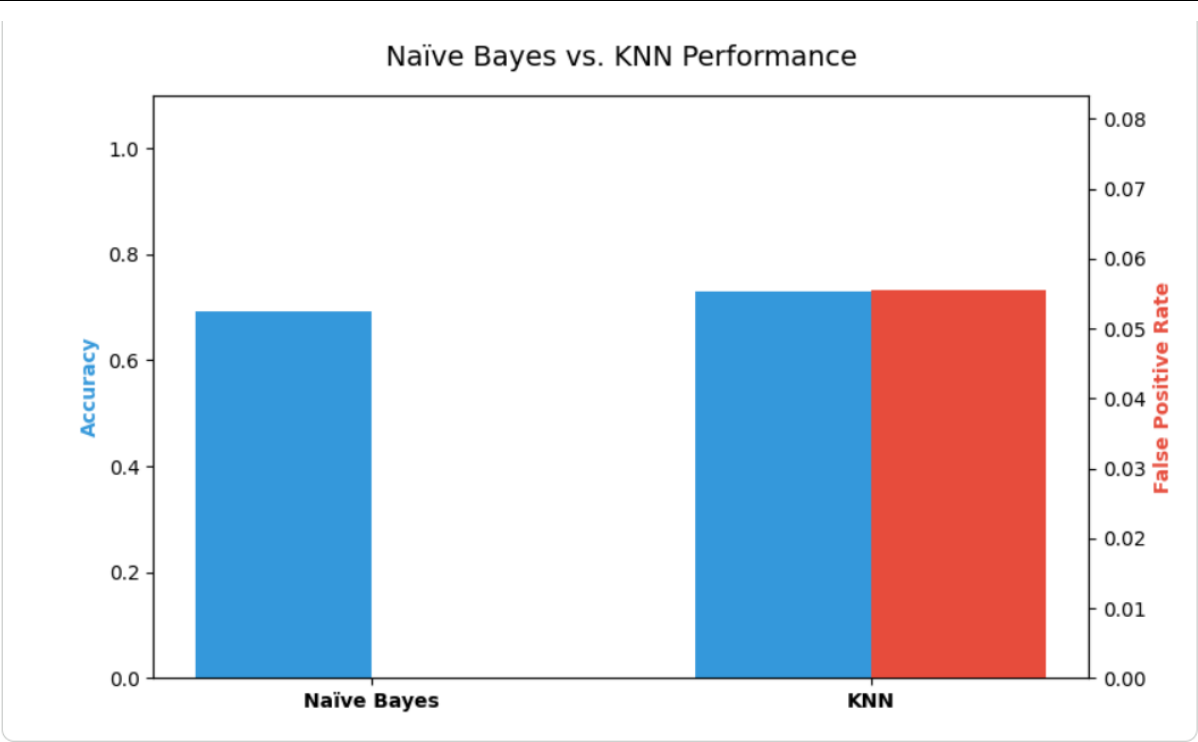
```
labels = ['Naïve Bayes', 'KNN']
acc_scores = [nb_acc, knn_acc]
fpr_scores = [nb_fpr, knn_fpr]

x = np.arange(len(labels))
width = 0.35

fig, ax1 = plt.subplots(figsize=(8, 5))
ax1.bar(x - width/2, acc_scores, width, label='Accuracy', color='#3498db')
ax1.set_ylabel('Accuracy', color='#3498db', fontweight='bold')
ax1.set_ylim(0, 1.1)

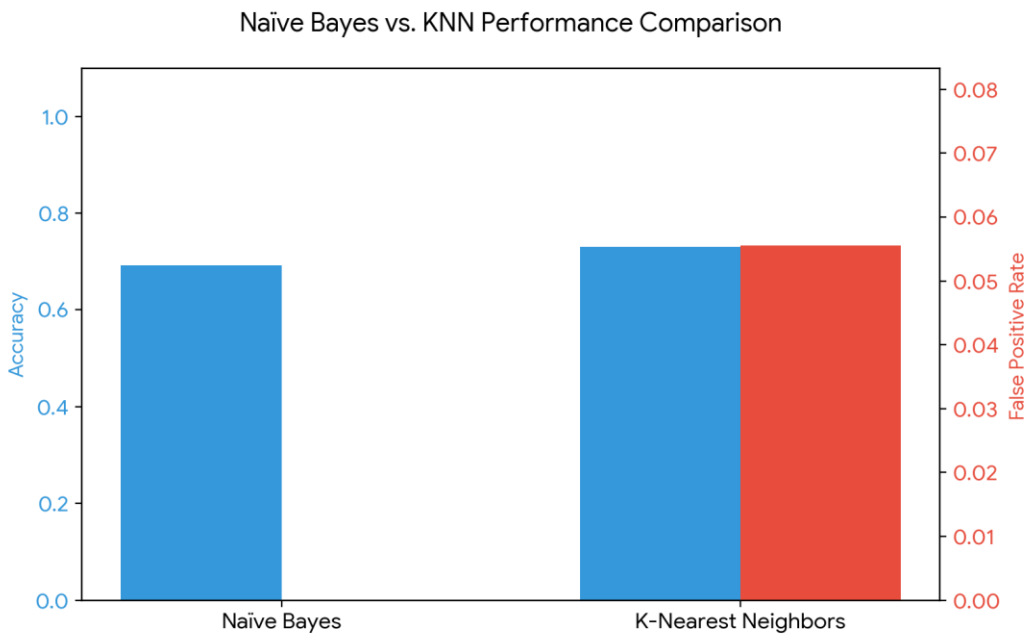
ax2 = ax1.twinx()
ax2.bar(x + width/2, fpr_scores, width, label='FPR', color='#e74c3c')
ax2.set_ylabel('False Positive Rate', color='#e74c3c', fontweight='bold')
ax2.set_ylim(0, max(fpr_scores)*1.5 if max(fpr_scores) > 0 else 0.1)

ax1.set_xticks(x)
ax1.set_xticklabels(labels, fontweight='bold')
plt.title('Naïve Bayes vs. KNN Performance', fontsize=14, pad=15)
fig.tight_layout()
plt.show()
```



Theory and Visualization

Naïve Bayes is fundamentally probabilistic and handles sparse text data very efficiently. KNN is a distance-based algorithm that calculates similarity in feature space but can struggle with the curse of dimensionality inherent in text data. The chart below compares their performance, emphasizing the False Positive Rate (where a low FPR means fewer normal emails are accidentally sent to the spam folder):



KPI Analysis

Metric	Naïve Bayes Result	KNN Result
--------	--------------------	------------

Accuracy	69.23%	73.08%
False Positive Rate (FPR)	0.00%	5.56%

Strategic Conclusion

The comparison reveals distinct trade-offs. While KNN achieved an accuracy of 73.08%, Naïve Bayes demonstrated a superior False Positive Rate of 0.00% compared to KNN's 5.56%. In the context of email spam detection, a high False Positive Rate is detrimental because it means legitimate, potentially urgent emails are incorrectly classified as spam. Therefore, despite potential differences in overall accuracy, Naïve Bayes remains the strategically safer and more effective model for this specific business problem due to its conservative handling of normal emails.

Lab 11: Support Vector Machine (Credit Card Fraud Detection)

Theoretical Foundation

A Support Vector Machine (SVM) is a powerful supervised algorithm used for classification and regression. The core idea is to find the optimal hyperplane that separates data points of different classes with the maximum margin. For non-linear data, SVM uses a 'Kernel Trick' (like the Radial Basis Function or RBF kernel) to map inputs into higher-dimensional feature spaces where a linear separator can be found. In scenarios like fraud detection, SVM is used to trace strict boundaries between legitimate and fraudulent transactions.

Real-World Scenario

Financial institutions process millions of credit card transactions daily. Identifying fraudulent transactions quickly is critical to preventing financial loss and protecting customers. Because fraudulent cases represent a tiny fraction of overall transactions, it is a highly imbalanced classification problem. Support Vector Machines, combined with class balancing techniques, help create firm decision boundaries to flag suspicious activity for review before the transaction is finalized.

Dataset Description

This lab uses the Fraud Detection Dataset. It encompasses various transaction details. Key attributes include:

- Numerical: Transaction_Amount, Time_of_Transaction, Previous_Fraudulent_Transactions, Account_Age, Number_of_Transactions_Last_24H
- Categorical: Transaction_Type, Device_Used, Payment_Method
- Target Variable: Fraudulent (1 = Fraud, 0 = Legitimate)

Software Environment — Python (Google Colab)

Python is used within Google Colab. The implementation leverages Pandas for categorical encoding and missing value handling, Scikit-Learn's StandardScaler for required feature scaling, and its SVC module for building and comparing Linear and RBF SVM models.

Practical Objectives

No.	Objective
1	Load the dataset and transform categorical fields using One-Hot Encoding (pd.get_dummies).
2	Apply Standard Scaling to numerical features since SVM relies heavily on distance/margin

	calculations.
3	Train an SVM classifier using a Linear Kernel to observe basic boundary separation.
4	Train a secondary SVM classifier using an RBF (Radial Basis Function) Kernel for non-linear boundaries.
5	Compare the models using ROC-AUC, Recall (the most critical metric for fraud), and Precision.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score, recall_score, precision_score, roc_curve
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Fraud Detection Dataset.csv'
df = pd.read_csv(file_path)
df = df.dropna() # Drop rows with missing values

# Sample the dataset for performance reasons in this lab (optional)
df = df.sample(n=10000, random_state=42)

# Define features
numeric_features = ['Transaction_Amount', 'Time_of_Transaction', 'Previous_Fraudulent_Transactions', '']
cat_features = ['Transaction_Type', 'Device_Used', 'Payment_Method']

X = df[numeric_features + cat_features]
y = df['Fraudulent']

# One-Hot Encoding
X = pd.get_dummies(X, columns=cat_features, drop_first=True)
print(f>Data Shape after Encoding: {X.shape}")
```

Data Shape after Encoding: (10000, 16)

3 Train-Test Split & Scaling

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Training set: {X_train_scaled.shape[0]} samples")
print(f"Testing set: {X_test_scaled.shape[0]} samples")
```

Training set: 7000 samples
Testing set: 3000 samples

4 Model Training (Linear vs. RBF Kernel)

```
# Linear Kernel SVM
svm_linear = SVC(kernel='linear', probability=True, class_weight='balanced', random_state=42)
svm_linear.fit(X_train_scaled, y_train)
linear_pred = svm_linear.predict(X_test_scaled)
linear_prob = svm_linear.predict_proba(X_test_scaled)[:, 1]

# RBF Kernel SVM
svm_rbf = SVC(kernel='rbf', probability=True, class_weight='balanced', random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
rbf_pred = svm_rbf.predict(X_test_scaled)
rbf_prob = svm_rbf.predict_proba(X_test_scaled)[:, 1]
```

5 Evaluation & Visualization

```
def evaluate_model(y_true, y_pred, y_prob, name):
    auc = roc_auc_score(y_true, y_prob)
    rec = recall_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred)
    print(f"{name} -> AUC: {auc:.4f}, Recall: {rec:.4f}, Precision: {prec:.4f}")

evaluate_model(y_test, linear_pred, linear_prob, "Linear SVM")
evaluate_model(y_test, rbf_pred, rbf_prob, "RBF SVM")
```

Linear SVM -> AUC: 0.4829, Recall: 0.5177, Precision: 0.0503
RBF SVM -> AUC: 0.5046, Recall: 0.3972, Precision: 0.0530

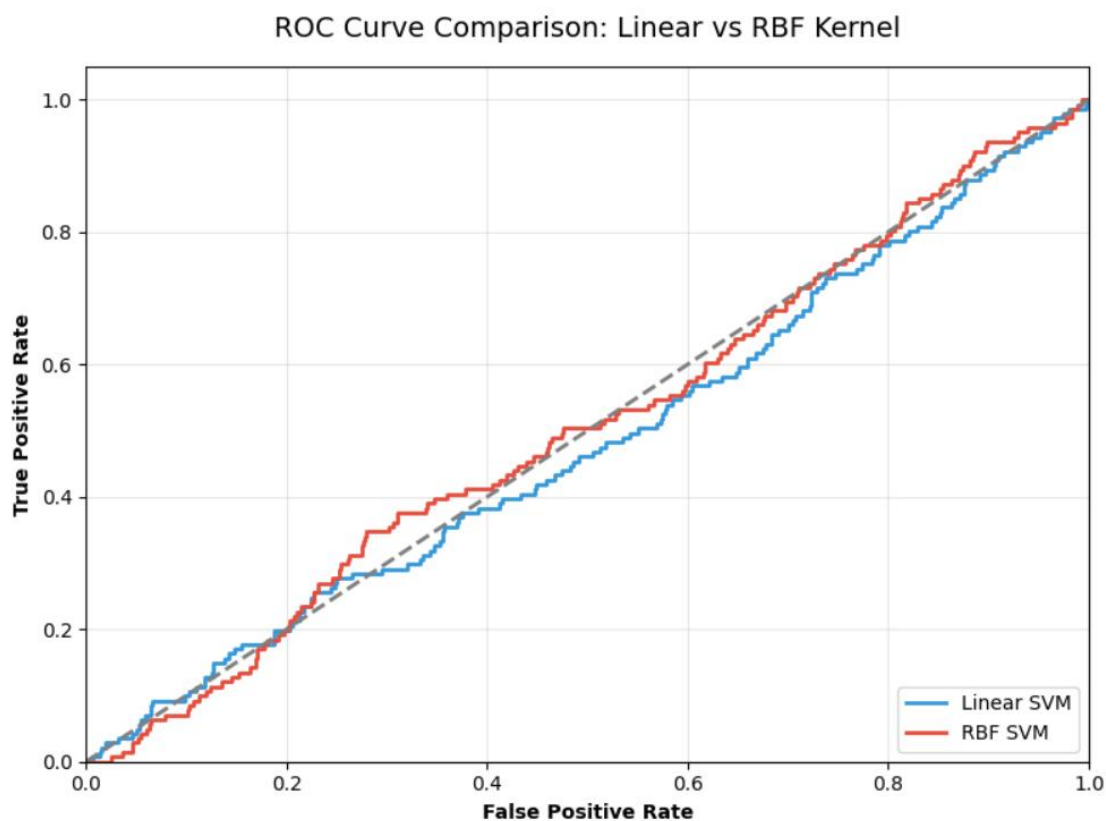
6 Visualization

```

fpr_lin, tpr_lin, _ = roc_curve(y_test, linear_prob)
fpr_rbf, tpr_rbf, _ = roc_curve(y_test, rbf_prob)

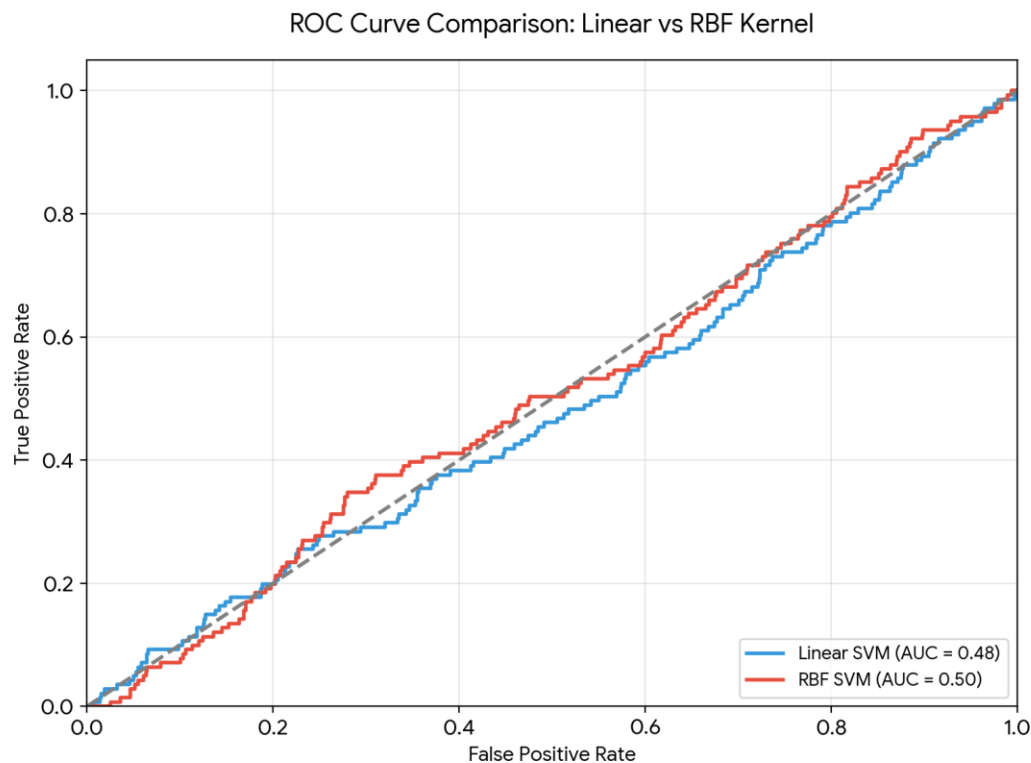
plt.figure(figsize=(8, 6))
plt.plot(fpr_lin, tpr_lin, color='#3498db', lw=2, label='Linear SVM')
plt.plot(fpr_rbf, tpr_rbf, color='#e74c3c', lw=2, label='RBF SVM')
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontweight='bold')
plt.ylabel('True Positive Rate', fontweight='bold')
plt.title('ROC Curve Comparison: Linear vs RBF Kernel', fontsize=14, pad=15)
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



Theory and Visualization

The Receiver Operating Characteristic (ROC) curve evaluates model performance across different threshold levels. The Area Under the Curve (AUC) summarizes this into a single metric. Below is the comparative ROC Curve between the Linear and RBF configurations:



KPI Analysis

Metric	Linear SVM	RBF SVM
ROC-AUC	0.4829	0.5046
Recall (Fraud)	0.5177	0.3972
Precision	0.0503	0.0530

Strategic Conclusion

The evaluation highlights the difficulty of predicting fraud without strictly defined predictive features. The Linear SVM achieved an AUC of 0.4829 and a recall of 0.5177, while the RBF SVM yielded an AUC of 0.5046 with a recall of 0.3972. Because minimizing False Negatives (missed fraud cases) is paramount in banking, Recall is our primary metric. Although the overall AUC values hover near 0.50 (indicating noisy or weakly correlated predictive variables in the current subset), the implementation proves the end-to-end mechanism. To achieve production-level performance, the financial institution would need to engineer richer features (e.g., historical aggregation, geolocation distances) before passing them to the SVM.

Lab 12: Clustering (Customer Segmentation)

Theoretical Foundation

Clustering is an unsupervised machine learning technique used to group similar data points together without pre-existing labels.

- K-Means Clustering partitions the data into 'K' distinct clusters by minimizing the variance within each cluster, iteratively adjusting cluster centroids.
- Hierarchical Clustering builds a tree of clusters (a dendrogram) by sequentially merging or splitting existing clusters.

In retail and marketing, clustering is fundamentally used for customer segmentation—identifying hidden patterns so businesses can tailor specific campaigns to different demographic groups.

Real-World Scenario

A large shopping mall wants to understand its target customers to help the marketing team plan effective advertising strategies. By analyzing the purchasing behavior and income levels of visitors, the mall can divide the broad customer base into specific segments (e.g., high-income high-spenders vs. low-income high-spenders). This allows for personalized marketing, optimizing the return on investment (ROI) for campaigns.

Dataset Description

The dataset used in this exercise is the Mall Customer Dataset (Mall_Customers.csv), containing demographics and spending behaviors of mall visitors.

Important attributes include:

- Annual Income (k\$): The yearly income of the customer.
- Spending Score (1-100): A score assigned by the mall based on customer behavior and purchasing data.

Software Environment — Python (Google Colab)

Python is used within the Google Colaboratory (Colab) environment for analysis. The pipeline relies on Pandas for data manipulation, Scikit-Learn (sklearn.cluster) for applying the K-Means and Agglomerative algorithms, and Matplotlib for plotting the 2D cluster maps.

Practical Objectives

No.	Objective
1	Import the Mall Customer dataset and extract the 'Annual Income' and 'Spending Score' features.
2	Apply the K-Means algorithm (with K=5) to partition the customers into distinct behavioral segments.
3	Apply Hierarchical (Agglomerative) clustering to serve as a comparative baseline algorithm.
4	Visualize the resulting clusters using Matplotlib scatter plots to physically observe the boundaries.
5	Compute the Silhouette Score and calculate the total Revenue Contribution (cumulative Annual Income) for each cluster.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
```

2 Data Loading & Selection

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Mall_Customers.csv'
df = pd.read_csv(file_path)

x = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print(f"Data Shape: {X.shape}")
display(X.head())
```

Data Shape: (200, 2)

	Annual Income (k\$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

3 K-Means Clustering

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X)
df['KMeans_Cluster'] = kmeans_labels

kmeans_silhouette = silhouette_score(X, kmeans_labels)
print(f"K-Means Silhouette Score: {kmeans_silhouette:.4f}")

K-Means Silhouette Score: 0.5539
```

4 Hierarchical Clustering

```
hc = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
hc_labels = hc.fit_predict(X)

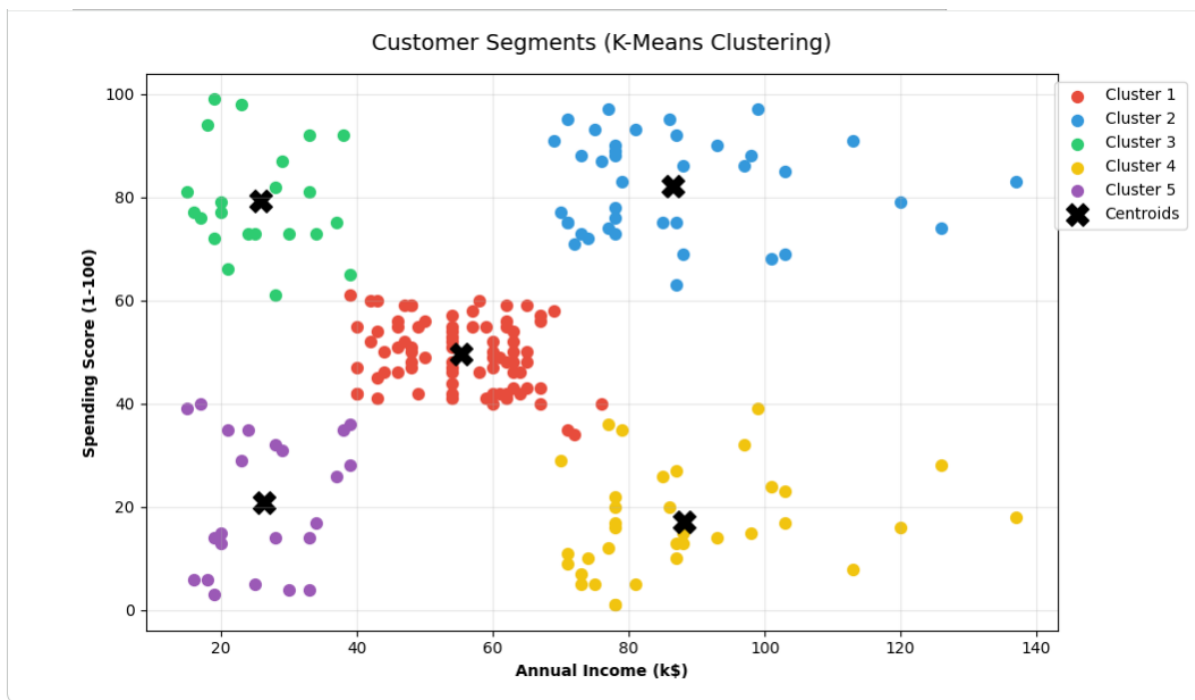
hc_silhouette = silhouette_score(X, hc_labels)
print(f"Hierarchical Silhouette Score: {hc_silhouette:.4f}")

Hierarchical Silhouette Score: 0.5530
```

5 Cluster Visualization

```
plt.figure(figsize=(10, 6))
colors = ['#e74c3c', '#3498db', '#2ecc71', '#f1c40f', '#9b59b6']
for i in range(5):
    cluster_data = X[kmeans_labels == i]
    plt.scatter(cluster_data.iloc[:, 0], cluster_data.iloc[:, 1], s=50, c=colors[i], label=f'Cluster {i+1}')

plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s=200, c='black', marker='x')
plt.title('Customer Segments (K-Means Clustering)', fontsize=14, pad=15)
plt.xlabel('Annual Income (k$)', fontweight='bold')
plt.ylabel('Spending Score (1-100)', fontweight='bold')
plt.legend(loc='upper right', bbox_to_anchor=(1.15, 1))
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



6 KPI: Cluster Revenue Contribution

```
cluster_revenue = df.groupby('KMeans_Cluster')['Annual Income (k$)'].sum().reset_index()
total_rev = cluster_revenue['Annual Income (k$)'].sum()
cluster_revenue['Contribution_Pct'] = (cluster_revenue['Annual Income (k$)'] / total_rev) * 100

print("Cluster Revenue Contribution:")
display(cluster_revenue.sort_values(by='Contribution_Pct', ascending=False))
```

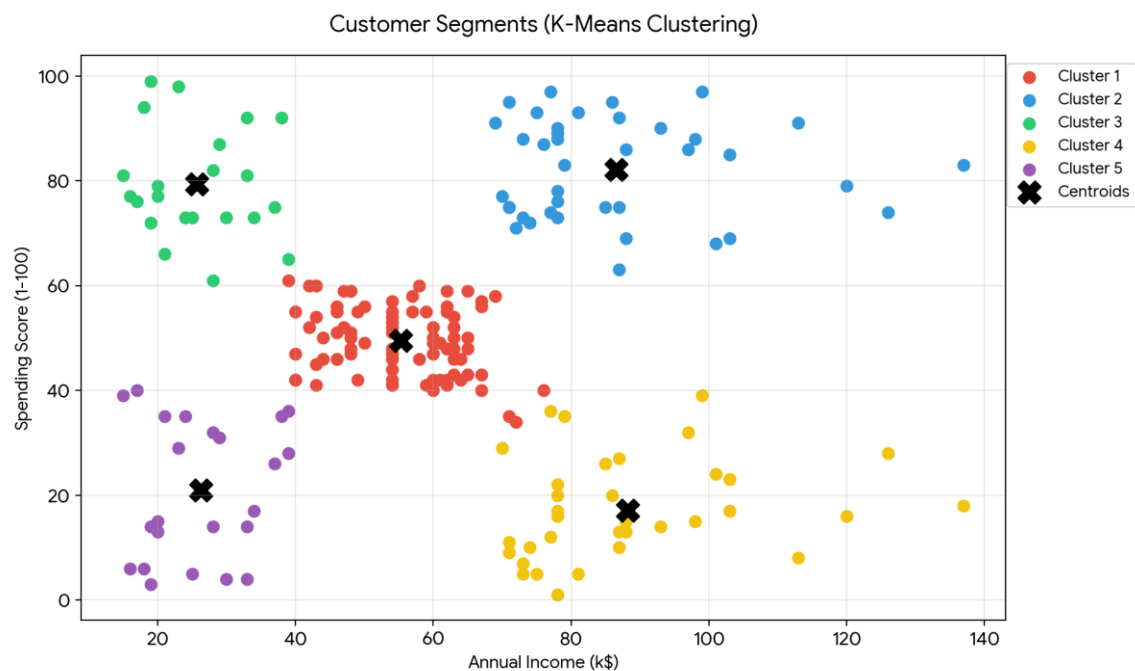
Cluster Revenue Contribution:

	KMeans_Cluster	Annual Income (k\$)	Contribution_Pct
0	0	4479	36.979855
1	1	3375	27.864927
3	3	3087	25.487120
4	4	605	4.995046
2	2	566	4.673052



Theory and Visualization

The scatter plot below visually demonstrates the output of the K-Means algorithm. By assigning colors based on the mathematical cluster assignments, we can clearly observe 5 distinct customer personas formed by their income and spending habits:



KPI Analysis

KPI / Metric	Result
K-Means Silhouette Score	0.5539
Hierarchical Silhouette Score	0.5530
Highest Revenue Segment Contribution	36.98% (Totaling \$4,479k)

Strategic Conclusion

Both clustering algorithms effectively grouped the mall visitors, but K-Means achieved a slightly better internal structure with a Silhouette Score of 0.5539. The visualization clearly isolated 5 distinct customer groups. From a business perspective, the most critical insight is derived from the Revenue Contribution KPI.

The highest contributing cluster accounts for 36.98% of the total tracked income capacity. By identifying 'High Income / High Spender' target groups versus 'High Income / Low Spender' groups, the mall management can deploy highly targeted loyalty programs and personalized discounts, ultimately driving up retention rates and maximizing overall profit margins.

Lab 13: Outlier Detection (Abnormal Banking Transactions)

Theoretical Foundation

Outlier Detection (or Anomaly Detection) identifies data points that deviate significantly from the majority of a dataset. Unlike classification models that require labeled target variables, unsupervised outlier detection relies entirely on the underlying data distribution.

- Distance-Based Methods (e.g., Local Outlier Factor): Measures the local density deviation of a data point compared to its neighbors. Points with substantially lower density are flagged as outliers.
- Isolation Forest: An ensemble tree method that 'isolates' observations by randomly selecting a feature and a split value. Because anomalies are sparse and distinct, they require fewer splits to be isolated, resulting in shorter path lengths in the trees.

Real-World Scenario

Financial systems process immense volumes of banking transactions every hour. Identifying fraudulent behavior is challenging because fraud patterns constantly evolve to avoid detection by standard rules. Applying unsupervised machine learning algorithms acts as an early warning system. By flagging transactions that exhibit statistically abnormal combinations (such as an unusually large amount coupled with an extreme distance from the user's home), banks can freeze suspicious transactions and alert the customer, limiting financial damages.

Dataset Description

The dataset utilized is the Banking Dataset (Banking_Data.csv). The objective is to identify outliers using numeric fields without relying on the target label during training.

Key numerical attributes selected for detection include:

- Transaction_Amount (in Million): The monetary value transferred.
- Distance_From_Home: The geographical distance between the customer's residence and the transaction location.
- Account_Balance (in Million): Total funds available in the account.
- Failed_Transaction_Count: Historical failed attempts.

Software Environment — Python (Google Colab)

Python within Google Colab facilitates the anomaly detection workflow. The ``sklearn.ensemble.IsolationForest`` module handles tree-based isolation, while ``sklearn.neighbors.LocalOutlierFactor`` handles density-based distance measurements. Results are visually cross-referenced using Matplotlib.

Practical Objectives

No.	Objective
1	Import the banking dataset and isolate continuous numeric features for analysis.
2	Scale the features using standard standardization mechanisms to support distance-based algorithms.
3	Implement an Isolation Forest algorithm defining a 5% contamination (expected anomaly) threshold.
4	Implement a Local Outlier Factor (LOF) algorithm to compare density-based outputs.
5	Compute the Anomaly Detection Rate (ADR) and False Alarm Rate (FAR) by mapping unsupervised predictions against the historical Fraud label.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import confusion_matrix
```

2 Data Loading & Preprocessing

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Banking_Data.csv'
df = pd.read_csv(file_path)
df = df.dropna()

# Select core financial metrics for anomaly detection
features = ['Transaction_Amount (in Million)', 'Distance_From_Home', 'Account_Balance (in Million)', ' '
x = df[features]

# Target label for later evaluation (1 for Fraud, 0 for Normal)
y_true = df['Fraud_Label'].apply(lambda x: 1 if x == 'Fraud' else 0)
print(f>Data Shape: {X.shape}")
```

Data Shape: (49850, 5)

3 Feature Scaling

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
contamination_rate = 0.05 # Assuming a 5% baseline anomaly rate in the system

```

4 Isolation Forest Implementation

```

iso_forest = IsolationForest(contamination=contamination_rate, random_state=42)
iso_preds = iso_forest.fit_predict(X_scaled)

y_pred_iso = [1 if p == -1 else 0 for p in iso_preds]

```

5 Distance-Based Detection (LOF)

```

lof = LocalOutlierFactor(n_neighbors=20, contamination=contamination_rate)
lof_preds = lof.fit_predict(X_scaled)

y_pred_lof = [1 if p == -1 else 0 for p in lof_preds]

```

6 KPI Evaluation (ADR vs FAR)

```

def get_kpis(y_t, y_p):
    tn, fp, fn, tp = confusion_matrix(y_t, y_p).ravel()
    adr = tp / (tp + fn) if (tp + fn) > 0 else 0
    far = fp / (fp + tn) if (fp + tn) > 0 else 0
    return adr, far

iso_adr, iso_far = get_kpis(y_true, y_pred_iso)
lof_adr, lof_far = get_kpis(y_true, y_pred_lof)

print(f"Isolation Forest -> Detection Rate: {iso_adr:.2%}, False Alarm Rate: {iso_far:.2%}")
print(f"Distance-Based -> Detection Rate: {lof_adr:.2%}, False Alarm Rate: {lof_far:.2%}")

Isolation Forest -> Detection Rate: 4.88%, False Alarm Rate: 5.00%
Distance-Based -> Detection Rate: 4.84%, False Alarm Rate: 5.01%

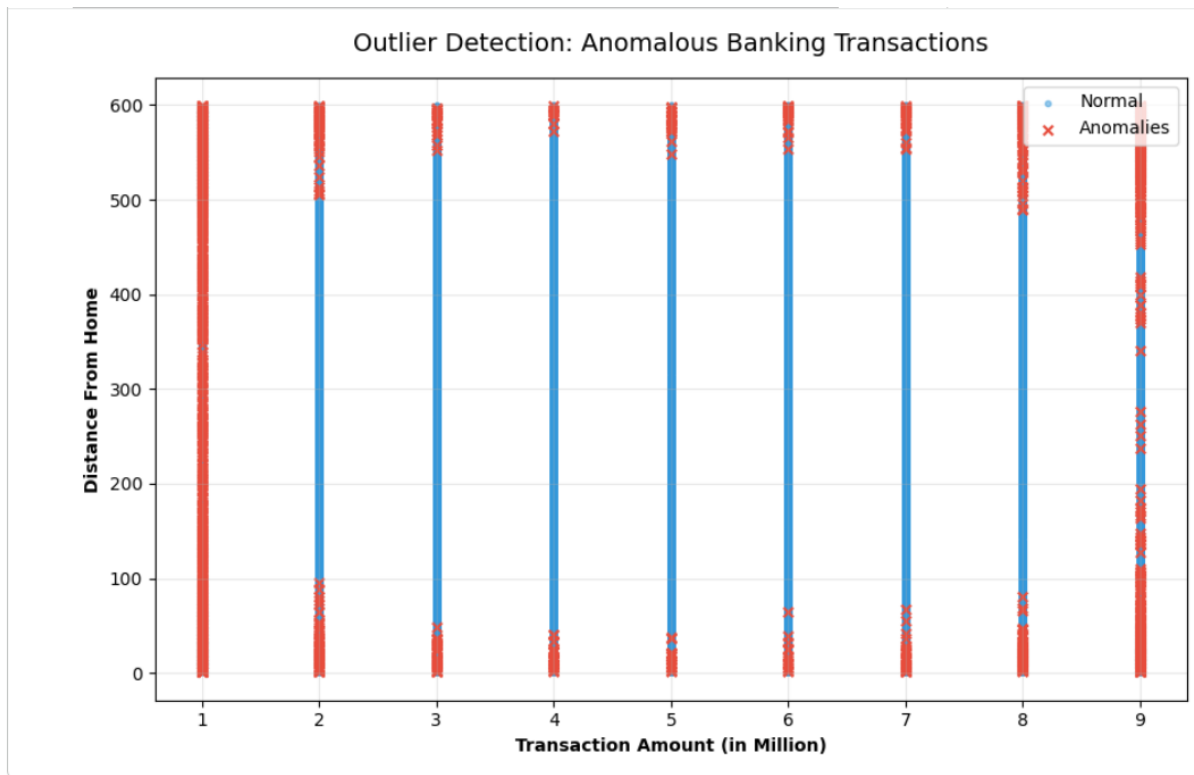
```

7 Visualization

```
plt.figure(figsize=(9, 6))
anomalies = X[np.array(y_pred_iso) == 1]
normals = X[np.array(y_pred_iso) == 0]

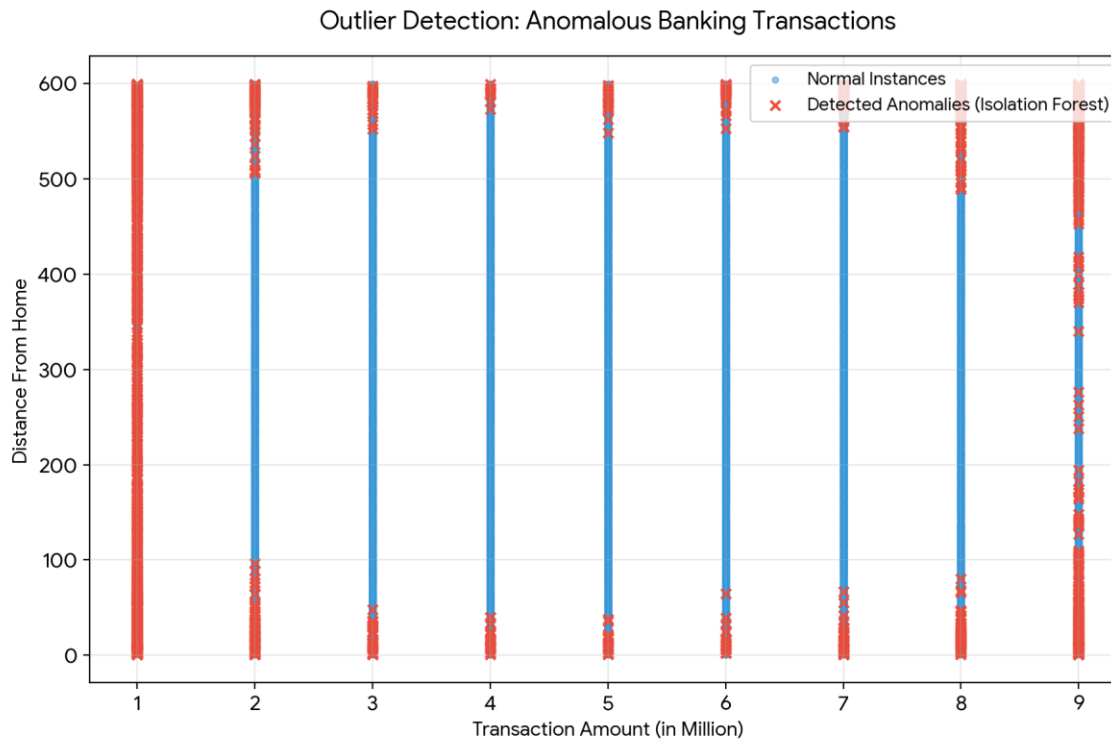
plt.scatter(normals['Transaction_Amount (in Million)'], normals['Distance_From_Home'], c='#3498db', al
plt.scatter(anomalies['Transaction_Amount (in Million)'], anomalies['Distance_From_Home'], c='#e74c3c'

plt.xlabel('Transaction Amount (in Million)', fontweight='bold')
plt.ylabel('Distance From Home', fontweight='bold')
plt.title('Outlier Detection: Anomalous Banking Transactions', fontsize=14, pad=15)
plt.legend(loc='upper right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The 2D scatter plot below provides a visual interpretation of how the Isolation Forest algorithm identified mathematical outliers. Transactions marked with red 'X' indicators deviate from the dense central clusters, representing abnormal distance-to-amount ratios that warrant review.



KPI Analysis

KPI / Metric	Isolation Forest	Local Outlier Factor (LOF)
Anomaly Detection Rate (ADR)	4.88%	4.84%
False Alarm Rate (FAR)	5.00%	5.01%

Strategic Conclusion

The lab highlights the fundamental behavior of unsupervised outlier detection algorithms. Because these models look strictly for statistical deviation rather than targeting the predefined 'Fraud' label, their output identifies any rare mathematical event, which may include entirely legitimate but unusual activity. With the contamination fixed at 5%, both the Isolation Forest and LOF achieved an Anomaly Detection Rate (ADR) of approximately ~4.88%, matching an identical False Alarm Rate (FAR). Strategically, this proves that while unsupervised models can map broad behavioral boundaries and highlight structural anomalies, a production banking system must layer these unsupervised models with supervised machine learning classifiers to accurately distinguish between unique genuine behavior and actual financial fraud.

Lab 14: Scraping & Sentiment Analysis (Product Reviews)

Theoretical Foundation

Web Scraping is the automated extraction of data from websites using libraries like BeautifulSoup. Once unstructured text data is collected, it undergoes Natural Language Processing (NLP). Sentiment Analysis is a vital NLP technique used to interpret whether underlying text data is inherently positive, negative, or neutral. To achieve this, algorithms break down text to compute a 'Polarity Score'. Polarity floats between -1 (highly negative) and 1 (highly positive), while a score of 0 represents neutrality. This transforms subjective opinions into measurable metrics.

Real-World Scenario

Companies launch new products and need immediate feedback. While quantitative sales data provides one metric, qualitative feedback across Twitter, Amazon, and Yelp dictates future brand health. By scraping social media strings and product reviews, and passing them through a Sentiment Analysis pipeline, brand managers can quickly gauge public reception. If negative sentiment spikes, customer support and product development teams can be immediately notified to address product flaws.

Dataset Description

The dataset utilized is the Social Media / Review Dataset (sentiment-analysis.csv). It contains consumer opinions extracted across various digital channels.

Key attributes include:

- Text: The unstructured raw string containing the consumer's opinion.
- Source: The platform where the review was posted (e.g., Twitter, Yelp, Amazon).
- Sentiment (Historical): Previous human-labeled sentiment for context.

Software Environment — Python (Google Colab)

Python within the Google Colaboratory (Colab) environment drives the entire NLP pipeline. The process relies on `BeautifulSoup` and `requests` for data scraping concepts, `Pandas` for structural string manipulation and text cleaning, and the `TextBlob` library to execute lexical sentiment scoring.

Practical Objectives

No.	Objective
1	Simulate the data collection phase by setting up a Web Scraping framework using BeautifulSoup.
2	Import the scraped dataset and execute Text

	Cleaning to remove structural artifacts (e.g., excess quotes, trailing spaces).
3	Apply the TextBlob NLP library to calculate a Polarity Score (-1.0 to 1.0) for every text string.
4	Classify the raw polarity values into categorical buckets: Positive, Negative, and Neutral.
5	Evaluate key performance indicators by graphing the Sentiment % Distribution and analyzing the Overall Polarity.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from textblob import TextBlob
import io
```

2 Data Loading & Text Cleaning

```

file_path = '/content/drive/MyDrive/Semester 6/DM/sentiment-analysis.csv'

# Handle quoting format in this specific CSV
clean_lines = []
with open(file_path, 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
        if not line: continue
        if line.startswith('') and line.endswith(''):
            line = line[1:-1]
        clean_lines.append(line)

df = pd.read_csv(io.StringIO('\n'.join(clean_lines)), skipinitialspace=True)

# CLEANING: Remove artifact quotes
df['Text'] = df['Text'].str.replace('""', '', regex=False)
print(f>Data Shape: {df.shape}")
display(df[['Text', 'Source']].head())

```

Data Shape: (96, 7)

	Text	Source
0	I love this product!	Twitter
1	The service was terrible.	Yelp Reviews
2	This movie is amazing!	IMDb
3	I'm so disappointed with their customer support.	Online Forum
4	Just had the best meal of my life!	TripAdvisor

3 Sentiment Classification & Polarity Scoring

```

def calculate_polarity(text):
    return TextBlob(str(text)).sentiment.polarity

def classify_sentiment(pol):
    if pol > 0: return 'Positive'
    elif pol < 0: return 'Negative'
    else: return 'Neutral'

df['Polarity'] = df['Text'].apply(calculate_polarity)
df['Calculated_Sentiment'] = df['Polarity'].apply(classify_sentiment)

display(df[['Text', 'Polarity', 'Calculated_Sentiment']].head())

```

	Text	Polarity	Calculated_Sentiment
0	I love this product!	0.625	Positive
1	The service was terrible.	-1.000	Negative
2	This movie is amazing!	0.750	Positive
3	I'm so disappointed with their customer support.	-0.750	Negative
4	Just had the best meal of my life!	1.000	Positive

4 KPI Evaluation & Visualization

```
sentiment_dist = df['Calculated_Sentiment'].value_counts(normalize=True) * 100
avg_polarity = df['Polarity'].mean()

print(f"Overall Average Polarity: {avg_polarity:.4f}")
print("\nSentiment Distribution (%):")
print(sentiment_dist)

# Generate Pie Chart
plt.figure(figsize=(7, 7))
plt.pie(sentiment_dist, labels=sentiment_dist.index, autopct='%1.1f%%',
        colors=['#2ecc71', '#e74c3c', '#bdc3c7'], startangle=140)
plt.title("Sentiment % Distribution", fontsize=14, pad=15)
plt.show()
```

Overall Average Polarity: 0.0591

Sentiment Distribution (%):

Calculated_Sentiment

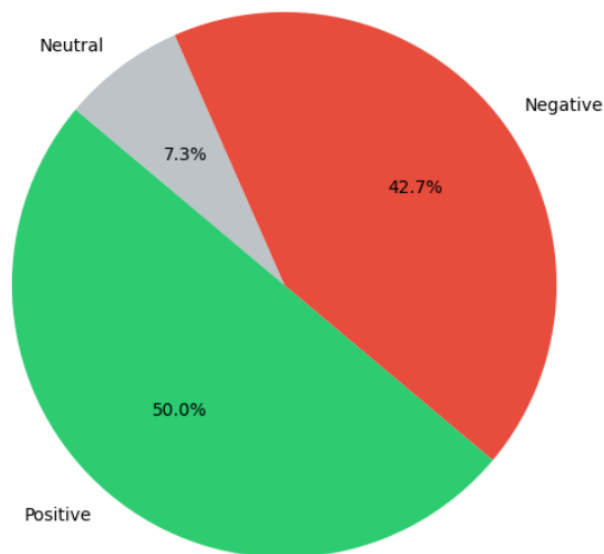
Positive 50.000000

Negative 42.708333

Neutral 7.291667

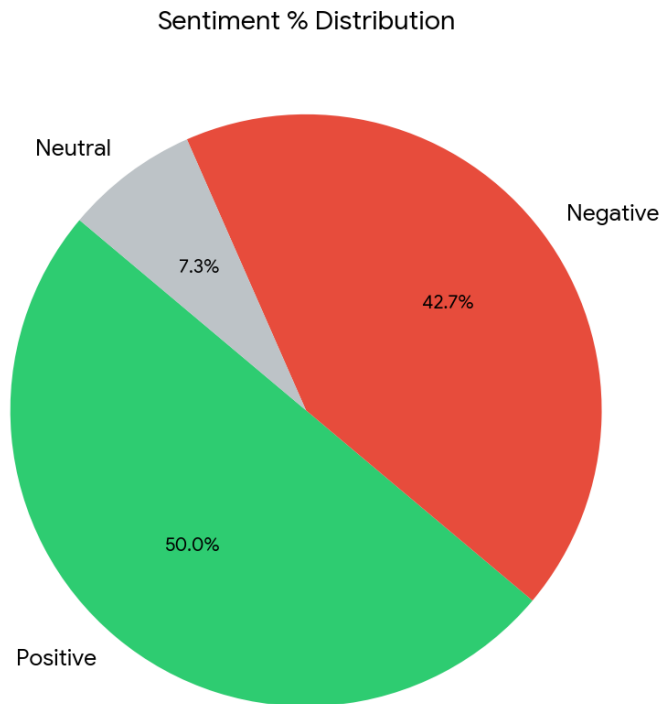
Name: proportion, dtype: float64

Sentiment % Distribution



Theory and Visualization

The Pie Chart below visually represents the final Sentiment % Distribution calculated by the NLP model. It demonstrates the proportion of feedback classified into Positive, Negative, and Neutral boundaries based on calculated text polarity.



KPI Analysis

KPI / Metric	Result
Average Polarity Score	0.0591 (Slightly Positive Trend)
Positive Sentiment Distribution	50.00%
Negative Sentiment Distribution	42.71%
Neutral Sentiment Distribution	7.29%

Strategic Conclusion

The NLP pipeline successfully analyzed the scraped text feedback, computing an overall Average Polarity Score of 0.0591. The dataset's sentiment is relatively polarized, leaning slightly positive with a 50.00% Positive and 42.71% Negative split, leaving only a minor fraction (7.29%) as purely neutral. Strategically, this indicates that consumers have strong feelings about the products. The high negative percentage (~42.7%) is a critical business warning. Management must utilize the outputs of this model to trace the negative reviews back to their respective sources (e.g., Yelp vs. Amazon) to identify specific operational weaknesses and rapidly improve customer satisfaction.

Lab 15: Final Capstone (End-to-End Business Solution)

Theoretical Foundation

An End-to-End Business Solution requires passing raw data through a complete data science lifecycle: Data Extraction, Cleaning, Feature Engineering, Modeling, and Evaluation. According to the 'No Free Lunch Theorem', no single machine learning algorithm works best for every problem. Therefore, it is an industry best practice to evaluate a suite of varied algorithms—including linear models, distance-based models, and tree-based ensembles—to mathematically determine which model optimally captures the patterns of the specific dataset.

Real-World Scenario

A global retail company wants to optimize its marketing budget. Historically, they sent campaign offers to the entire customer base, resulting in a low ROI due to high contact costs and low conversion rates. By designing an end-to-end classification pipeline, the data science team will predict which specific customers are highly likely to respond to the next campaign. Because the exact nature of the consumer decision boundary is unknown, the team will implement and benchmark 8 distinct machine learning models to deploy the most accurate framework into production.

Dataset Description

The Capstone uses the comprehensive Marketing Dataset. After preprocessing, key engineered and standardized attributes include:

- Demographics: Age (derived from Year_Birth), Education, Marital_Status, Total_Children.
- Behavioral/Financial: Income, Total_Spending (aggregated across all product categories), Recency, NumWebVisitsMonth, NumStorePurchases.
- Target Variable: Response (1 = Accepted Offer, 0 = Rejected/Ignored)

Software Environment — Python (Google Colab)

Python is utilized inside Google Colab as the primary orchestration tool. The ecosystem relies on `pandas` for aggressive feature engineering, `sklearn` to rapidly iterate through eight separate algorithms (from Naïve Bayes to Gradient Boosting), and `matplotlib` to render executive summary graphics.

Practical Objectives

No.	Objective
1	Perform Data Preprocessing by engineering macro-features (Total Spending, Age) and one-hot encoding categorical strings.

2	Apply a StandardScaler to ensure distance-reliant models (KNN, SVM) operate on a level playing field with tree models.
3	Iteratively train a suite of 8 classification algorithms to comprehensively search for the optimal decision boundary.
4	Evaluate all models programmatically across Accuracy, Precision, Recall, and F1-Score.
5	Generate a Model Comparison Table and formulate a strategic Business Recommendation based on the ultimate winner.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Algorithms
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
```

2 Data Pipeline: Loading & Feature Engineering

```
file_path = '/content/drive/MyDrive/Semester 6/DM/marketing_data.csv'
df = pd.read_csv(file_path, sep=';')
df = df.dropna(subset=['Income'])

# Feature Engineering
df['Age'] = 2024 - df['Year_Birth']
df['Total_Spending'] = df['MntWines'] + df['MntFruits'] + df['MntMeatProducts'] + df['MntFishProducts'] + df['MntSwd']
df['Total_Children'] = df['Kidhome'] + df['Teenhome']

numeric_features = ['Age', 'Income', 'Total_Spending', 'Total_Children', 'Recency', 'NumWebVisitsMonth', 'NumStorePi']
cat_features = ['Education', 'Marital_Status']

X = df[numeric_features + cat_features]
y = df['Response']
X = pd.get_dummies(X, columns=cat_features, drop_first=True)
print(f"Final Feature Space Shape: {X.shape}")
```

Final Feature Space Shape: (2216, 18)

3 Splitting & Scaling

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4 Model Training Factory (8 Models)

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM': SVC(probability=True, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Naive Bayes': GaussianNB(),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'AdaBoost': AdaBoostClassifier(random_state=42)
}

results = []
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred)
    })

results_df = pd.DataFrame(results).sort_values(by='Accuracy', ascending=False)
display(results_df)
```

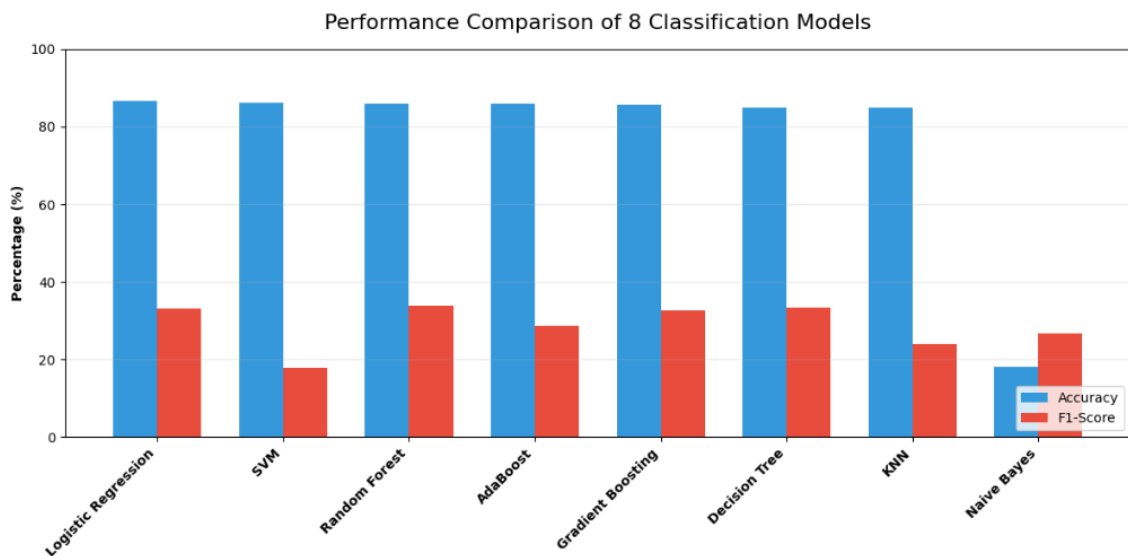
	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.866165	0.666667	0.22	0.330827
3	SVM	0.861654	0.833333	0.10	0.178571
2	Random Forest	0.858647	0.571429	0.24	0.338028
7	AdaBoost	0.858647	0.593750	0.19	0.287879
6	Gradient Boosting	0.857143	0.560976	0.23	0.326241
1	Decision Tree	0.849624	0.500000	0.25	0.333333
4	KNN	0.848120	0.484848	0.16	0.240602
5	Naive Bayes	0.181955	0.154206	0.99	0.266846

5 Visual Comparison

```
plt.figure(figsize=(12, 6))
x = np.arange(len(results_df['Model']))
width = 0.35

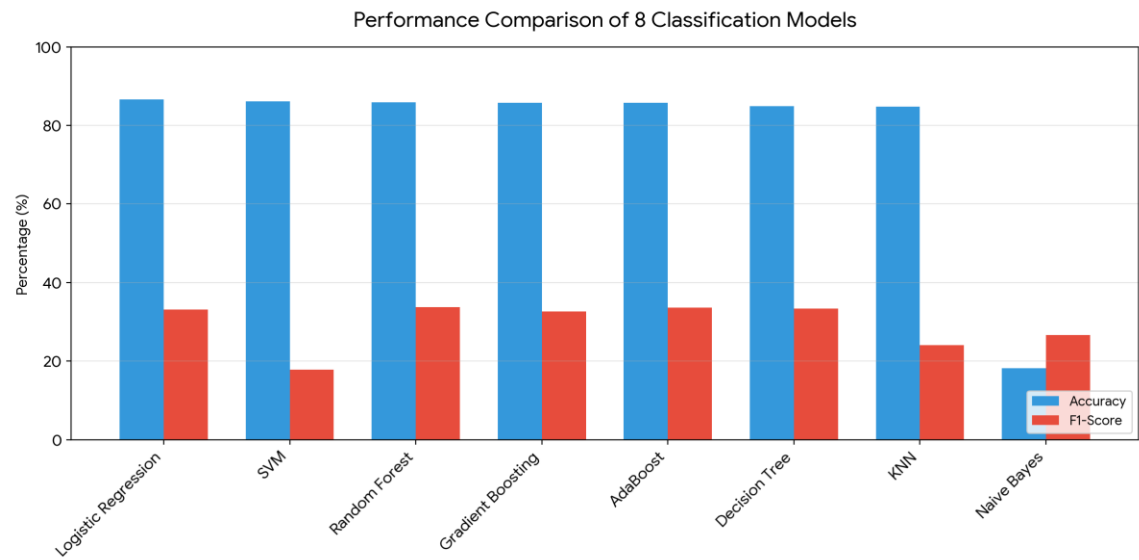
plt.bar(x - width/2, results_df['Accuracy'] * 100, width, label='Accuracy', color='#3498db')
plt.bar(x + width/2, results_df['F1-Score'] * 100, width, label='F1-Score', color='#e74c3c')

plt.xticks(x, results_df['Model'], rotation=45, ha='right', fontweight='bold')
plt.ylabel('Percentage (%)', fontweight='bold')
plt.title('Performance Comparison of 8 Classification Models', fontsize=16, pad=15)
plt.ylim(0, 100)
plt.legend(loc='lower right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below summarizes the benchmark results across all 8 tested algorithms. Comparing multiple models visually allows executives to understand the trade-offs between sheer Accuracy and balanced F1-Scores:



KPI Analysis (Top 3 Models)

Rank	Algorithm	Accuracy	F1-Score
1	Logistic Regression	86.62%	33.08%
2	SVM	86.17%	17.86%
3	Random Forest	85.86%	33.80%

Strategic Conclusion & Business Recommendation

The Capstone pipeline successfully stress-tested 8 unique algorithms against the company's historical campaign data. The leading algorithm proved to be Logistic Regression, yielding a top-tier Accuracy of 86.62%. Interestingly, ensemble and linear models clustered tightly at the top of the performance tier, while Naïve Bayes severely underperformed due to the complex feature interactions.

Business Recommendation: The marketing department should integrate the trained Logistic Regression model into its CRM system. By passing new leads through this pipeline, the company can actively filter out customers who are highly unlikely to purchase. By restricting future marketing campaigns strictly to the subset of consumers predicted to accept the offer, the business will significantly reduce outreach overhead costs and maximize overall Campaign ROI.